

実験ログ A: プロシージャルモデリング 基礎

2026-09-13 | 収録 18 件

hython で実際にノードを組み、その結果を実測して記録したもの。数値はすべて実行して得た値で、推測は含まない。

各実験は「何を作ったか」「ノードの接続」「変更したパラメータ」「結果」「わかったこと」「次にやること」の順で並んでいる。用語は別ソースの「用語集: Houdini 初心者向け」にまとめてある。

収録している実験

- Box を山にして細分割し、位置で色を付ける
- subdivide の回数で、形と数はどう変わるか
- mountain と subdivide は、順序を変えると何が変わるか
- mountain のノイズパラメータは、それぞれ何に効くのか
- basis (ノイズの種類) 14通りで、形はどこまで変わるか
- height は純粋な倍率か。gain と bias は何に効くか
- 平面で地形を作り、指標を作り直す
- scatter と copy to points — 地面に物を生やす
- attribute wrangle で VEX を書く — 何が何回動くのか
- for-each ループと VEX — 同じことをやって比べる
- crease で角を残す — どれだけの重みが要るのか
- polyextrude と boolean — 硬い形を作る2つの道具
- 時間軸への対応 — フレームを進めて記録する
- RBD 破壊 — 最初の本物のシミュレーション
- Vellum クロス — 布で保存されるべき量は何か
- Pyro 煙 — dissipation の正体を式で突き止める
- FLIP 液体 — 組み方が分からず保留
- Karma でのレンダリング — ノイズはサンプル数で本当に減るのか

Box を山にして細分割し、位置で色を付ける

2026-09-13 | Houdini 21.0.700 | /obj/box_mountain | ノード 5 / 接続 4

何を作ったか

プロシージャルモデリングの最小構成を確認するための実験。box SOP で立方体を作り、mountain SOP でノイズによる凹凸を与え、subdivide SOP で細分割し、color SOP で色を乗せた。最後に null ノードを OUT として置き、そこを出力点にしている。

Houdini の基本的な考え方である「ジオメトリを次のノードに流し込んで加工を積み重ねる」形をそのまま辿っている。各ノードは前のノードの出力だけを入力として受け取るため、途中のどのノードでも結果を確認できる。

ノードの接続

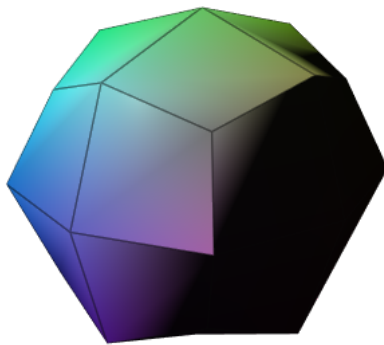


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
mountain1	mountain::2.0	elementsize = 0.8 height = 0.35
subdivide1	subdivide	(変更なし)
color1	color	colortype = 1 ramp2cb = 1.0 ramp2cg = 1.0 ramp2cr = 1.0 ramp2pos = 1.0

ノード	タイプ	デフォルトから変更したパラメータ
OUT	null	(変更なし)

結果



Houdini

OpenGL ROP による Smooth Wire Shaded

表示。ワイヤーフレームが乗っているため、ネットワークが生成したトポロジが確認できる。画像内のウォーターマークは Apprentice ライセンスによるもの。

ジオメトリ

ポイント数	26
プリミティブ数	24
バウンディングボックス	[-0.4794, -0.4768, -0.4767] ~ [0.4932, 0.4959, 0.4959]
ポイント属性	P, Cd
プリミティブ属性	なし
ディテール属性	なし

わかったこと

- ・ box は 8 ポイント / 6 プリミティブ。subdivide を 1 回かけると 26 ポイント / 24 プリミティブになる。Catmull-Clark 細分割なので、回数を増やすと立方体はどんどん球に近づく（2 回で既に球状になり、元が箱だと分からなくなった）。

- ・ mountain SOP

は既存のポイントを法線方向にノイズで押し出すもので、ポイントを増やさない。つまり box のままかけても 8 点しか動かないため、凹凸のディテールを出すには先に subdivide などで分割数を上げる必要がある。

- ・ color SOP の colortype は数値メニュー（0=Constant, 1=Bounding Box, 2=Random, 3=Ramp from Attribute, 4=Random from Attribute）。文字列で指定すると TypeError になる。今回は 1

を使い、バウンディングボックス内の位置をそのまま RGB に割り当てた。

- ・ color SOP を通すとポイント属性に Cd が追加される（P のみ → P, Cd）。属性が増えたかどうかはジオメトリの統計で確認できる。

次にやること

- ・ subdivide の回数を変えて、ポイント数と見た目がどう変わるかを並べて比較する
- ・ mountain を subdivide の後ろに移動し、分割後に凹凸をかけた場合との違いを見る
- ・ grid から地形を作る流れに広げ、プロシージャルモデリングの定番パターンに入る

subdivide の回数で、形と数はどう変わるか

2026-09-13 | Houdini 21.0.700 | /obj/subdiv_compare | ノード 5 / 接続 4

何を作ったか

実験001で「subdivideの回数を増やすと立方体が球に近づく」ことに気づいたので、それを数値で確かめる実験。box SOP に subdivide SOP

を4通り（0回・1回・2回・3回）ぶら下げ、すべて同じカメラでレンダして並べた。

同じカメラで撮っているのが重要な点で、各変種を自動でフレームに収めてしまうと、大きさの違いが消えて比較にならない。画を見ると球が立方体よりはっきり小さいことが分かるが、これは目の錯覚ではなく実測でも縮んでいる。

さらに「なぜ縮むのか」「どこまで縮むのか」「1回目だけ縮まないのはなぜか」を追加検証で確定させた。

ノードの接続

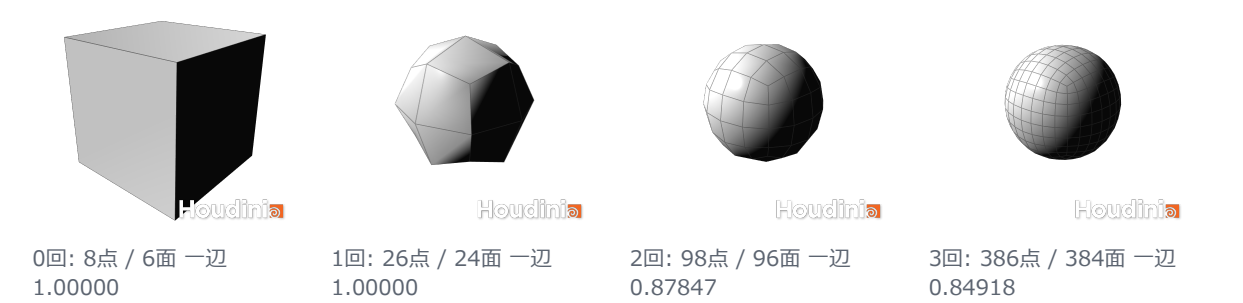


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
subdiv_0	subdivide	iterations = 0
subdiv_1	subdivide	(変更なし)
subdiv_2	subdivide	iterations = 2
subdiv_3	subdivide	iterations = 3

回数を変えた結果

すべて同じカメラ・同じ表示モード（Smooth Wire Shaded）。元の立方体の輪郭に対して、分割を重ねるほど内側に縮んでいくのが分かる。



iterations	ポイント数	プリミティブ数	面数の倍率	一辺の長さ
0	8	6	—	1.00000
1	26	24	4.00	1.00000

iterations	ポイント数	プリミティブ数	面数の倍率	一辺の長さ
2	98	96	4.00	0.87847
3	386	384	4.00	0.84918
4	1538	1536	4.00	0.84192
5	6146	6144	4.00	0.84011
6	24578	24576	4.00	0.83966

アルゴリズム別の比較（iterations = 3 で固定）

subdivide SOP の algorithm パラメータには5つの選択肢がある。既定値は 2（OpenSubdiv Catmull-Clark）。同じ3回分割で結果がどう違うかを測った。

algorithm	ポイント数	プリミティブ数	一辺の長さ
Houdini Catmull-Clark	386	384	0.84918
Mantra-Compatible Catmull-Clark	386	384	0.83951
OpenSubdiv Catmull-Clark（既定）	386	384	0.84918
OpenSubdiv Loop	386	768	0.86932
OpenSubdiv Bilinear	386	384	1.00000

回数を変えたときの一辺（2つのアルゴリズムの比較）

OpenSubdiv Catmull-Clark は回数を増やすほど値が動くが、Mantra-Compatible は回数を変えても一定。これが何を意味するかは「わかったこと」の最後に書いた。

iterations	OpenSubdiv CC	Mantra-Compatible
1	1.00000	0.83951
2	0.87847	0.83951
3	0.84918	0.83951
4	0.84192	0.83951
5	0.84011	0.83951

わかったこと

- iterations = 0 は素通し。8点 / 6面のまま何も起きない。比較の基準として使える。
- プリミティブ数は1回ごとに正確に4倍になる（6 → 24 → 96 → 384 → 1536 → 6144 → 24576）。四角形1枚が4枚に割れるため。3回で64倍、6回で4096倍なので、回数を安易に増やすと破綻する。
- ポイント数は常に「プリミティブ数 + 2」になっている。これは偶然ではなくオイラーの多面体定理から導ける。閉じた四角形メッシュでは各辺を2つの面が共有するので $E = 2F$ 、これを $V - E + F = 2$ に代入すると $V = F + 2$ 。実測した7通りすべてで一致した。
- 1回目だけ一辺が 1.00000 のまま縮まない。検証したところ、+X方向の端を保持しているポイントは (0.5, 0, 0) の1点だけだった。これは元の +X 面の中心そのもの。Catmull-Clark は各面の中心に新しい点を作るため、1回目はその新点元が元の面の上に乗り、バウンディングボックスが変わらない。2回目でその点も内側に引かれ 0.87847 に縮む。

- ・ 一辺の縮み方は等比的に収束する。差を取ると $0.02929 \rightarrow 0.00726 \rightarrow 0.00181 \rightarrow 0.00045$ で、毎回およそ $1/4$ （実測 $0.248 / 0.249 / 0.249$ ）。ここから外挿した極限值は約 0.83951 になる。

- ・ 縮みの原因は「分割」ではなく「平滑化」だと確定した。OpenSubdiv Bilinear（平滑化せず分割するだけの方式）で3回分割しても一辺は 1.00000 のまま変わらない。面数は同じ384なので、分割量は同じで平滑化の有無だけが違う。

- ・ OpenSubdiv Loop だけプリミティブ数が 768 になる（他は384）。Loop は三角形メッシュ向けの方式なので、四角形が三角形に割られて枚数が倍になる。四角形を保ちたい場面では選んではいけない。

- ・ Mantra-Compatible Catmull-Clark は回数を $1 \sim 5$ のどれにしても一辺が 0.83951 で一定だった。これは分割を繰り返して極限に近づけるのではなく、極限曲面上の位置を直接求めていることを示している。実際この値は、OpenSubdiv CC の収束列から外挿した極限值 0.83951 と一致した。つまり「無限回分割した先の形」を1回で出しているのがこのモード。

次にやること

- ・ crease（折り目）を使って角を保持できるか。立方体らしさを残したまま分割する方法があるはず

- ・ mountain と subdivide

の順序を入れ替えて、凹凸のディテールがどう変わるかを比較する（実験003）

- ・ polyextrude

など他のSOPでもポイント数・プリミティブ数の増え方を測り、重くなる原因を数で把握する

mountain と subdivide

は、順序を変えると何が変わるか

2026-09-13 | Houdini 21.0.700 | /obj/order_compare | ノード 14 / 接続 13

何を作ったか

実験001で「mountain はポイントを増やさないなので、細かい凹凸を出すには先に分割数を上げる必要がある」と書いた。これは筋は通っているが、実際に確かめてはいなかった。そこで順序違いを並べて比較し、違いを数字で出す。

比較したのは6通り。ノイズなしの基準形、ノイズを先にかけたもの、分割を先にしたもの、分割をさらに細かくしたもの、ノイズの特徴サイズを小さくしたもの、そのうえで点数を増やしたもの。すべて同じカメラでレンダした。

この実験では測り方を一度間違えている。最初に選んだ指標では違いが正しく出ず、途中で別の指標に切り替えた。その経緯も含めて記録した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
A_mountain	mountain::2.0	elementsizer = 0.8 height = 0.35
A_subdiv	subdivide	iterations = 2
B_subdiv	subdivide	iterations = 2
B_mountain	mountain::2.0	elementsizer = 0.8 height = 0.35
C_subdiv	subdivide	iterations = 3
C_mountain	mountain::2.0	elementsizer = 0.8 height = 0.35
D_subdiv	subdivide	iterations = 3
D_mountain	mountain::2.0	elementsizer = 0.25 height = 0.35
E_subdiv	subdivide	iterations = 4
E_mountain	mountain::2.0	elementsizer = 0.25 height = 0.35
base_subdiv	subdivide	iterations = 2
base3_subdiv	subdivide	iterations = 3
base4_subdiv	subdivide	iterations = 4

6通りの結果

すべて同じカメラ・同じ表示モード。左から右へ、凹凸がどう変わるかを見てほしい。mountainのパラメータは height = 0.35 で固定し、elementsize（ノイズの特徴サイズ）だけを D と E で 0.8 から 0.25 に変えている。



Houdini

基準: 分割のみ、ノイズなし 98点 / 96面



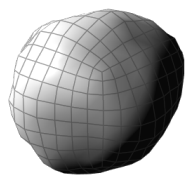
Houdini

A: mountain → subdiv(2) 98点 / 96面



Houdini

B: subdiv(2) → mountain 98点 / 96面



Houdini

C: subdiv(3) → mountain 386点 / 384面



Houdini

D: C と同じだが elementsize 0.25 / 386点



Houdini

E: D と同じノイズで 点数のみ増加 / 1538点

基準形からの点ごとのずれ

同じ分割回数ならポイントの順序も同じなので、ノイズなしの基準形と1点ずつ引き算できる。これが「ノイズがどれだけ形を動かしたか」を直接表す。A・B は2回分割の基準形と、C・D は3回分割の基準形と、E は4回分割の基準形と比べている。

変種	ポイント数	平均のずれ	最大のずれ
A: mountain → subdiv(2)	98	0.02069	0.03671
B: subdiv(2) → mountain	98	0.02479	0.08062
C: subdiv(3) → mountain	386	0.02504	0.08009
D: C + elementsize 0.25	386	0.03236	0.11985
E: D + 点数4倍	1538	0.03239	0.13173

最初を選んで失敗した指標（表面積）

凹凸が増えれば表面積も増えるはずだと考え、まず表面積を測った。ところが mountain を足すと表面積は減った。ノイズは外側にも内側にも押すため全体が縮み（一辺が 0.8785 から 0.8545

へ)、面積は大きさの2乗に比例するのでその影響が凹凸の効果を打ち消していた。一辺の2乗で割って正規化しても、値の大小が凹凸の量と対応しない。大きさと凹凸が混ざった指標だったということ。

変種	表面積	基準比	一辺	面積 / 一辺 ²
基準 (ノイズなし)	2.43164	1.000	0.8785	3.15076
A	2.30052	0.946	0.8545	3.15066
B	2.24947	0.925	0.8535	3.08797
C	2.16988	0.892	0.8314	3.13918
D	2.49922	1.028	0.9123	3.00282
E	2.66892	1.098	0.9278	3.10046

わかったこと

- ・順序を入れ替えてもポイント数とプリミティブ数は同じ (どちらも98点 / 96面)。つまりデータの重さは変わらない。変わるのは形だけ。
- ・最大のずれは A が 0.03671、B が 0.08062 で、B が約2.2倍大きい。同じ mountain の設定でも、分割したあとにかけた方が凹凸が残る。実験001で立てた見込みは正しかった。
- ・なぜそうなるかも測れた。box に mountain をかけた直後は、8点すべてが平均 0.03607 動いている。ところがそのあと subdivide をかけると、基準形からのずれは平均 0.02069・最大 0.03671 まで下がる。Catmull-Clark の平滑化は周囲の点を平均するので、せっかく動かした凹凸をなめしてしまう。
- ・C は B とほぼ同じずれだった (平均 0.02504 対 0.02479、最大 0.08009 対 0.08062)。点を4倍に増やしても凹凸の大きさは変わらない。点数はノイズの振幅を決めていない。
- ・振幅を決めていたのは elementsize だった。C と D は同じ386点だが、elementsiz を 0.8 から 0.25 に下げると平均のずれが 0.02504 から 0.03236 に増え、画も明確に岩ようになった。
- ・E は D と同じノイズ設定で点数だけ4倍にしたもの。平均のずれは 0.03236 から 0.03239 でほぼ変わらない。一方で画は、同じ起伏をより滑らかな面で表現できている。つまり点数は「ノイズをどれだけ忠実に表現できるか (解像度)」を決めていて、「起伏の大きさ (振幅)」は決めていない。
- ・ただし最大のずれだけは点数とともに少し増える (0.11985 → 0.13173)。点が増えるとノイズのより極端な場所まで拾えるようになるため、振幅そのものが増えたわけではない。
- ・実用的な結論: 細かいディテールが欲しいときは、分割数を上げるだけでは足りない。elementsiz も一緒に小さくする。そして mountain は分割のあとに置く。この3つが揃って初めて狙った凹凸になる。
- ・測り方についての反省。表面積のような「全体をひとつの数にまとめた量」は、大きさと形の変化が混ざってしまう。基準形と1点ずつ対応づけて引き算する方が、知りたいことを素直に測れた。比較実験では、まず基準形を作り、それと1対1で比べられる量を選ぶべき。

次にやること

- ・mountain の他のパラメータ (roughness, octaves) が振幅と細かさのどちらに効くのかを、同じやり方で切り分ける
- ・grid から地形を作り、平面の場合も同じ結論になるか確かめる (実験004)
- ・crease を使って、分割しても角を保持できるかを確かめる (実験002からの持ち越し)

mountain

のノイズパラメータは、それぞれ何に効くのか

2026-09-13 | Houdini 21.0.700 | /obj/mountain_params | ノード 15 / 接続 14

何を作ったか

実験003で「点数は解像度を決め、elementsize は振幅を決める」と分かった。では残りのノイズパラメータは何を変えているのか。同じ切り分け方で調べる。

最初に分かったのは名前の問題だった。パラメータ名は roughness や octaves ではなく rough / oct / lac。実際に mountain::2.0 が持っている名前を確認してから使わないと、そもそも動かない。

この実験でも測り方を追加している。003で使った「基準形からのずれ」だけでは、これらのパラメータをほとんど区別できなかった。振幅ではなく形の性質を変えるパラメータだったため、性質を測る指標を新しく用意した。

ノードの接続



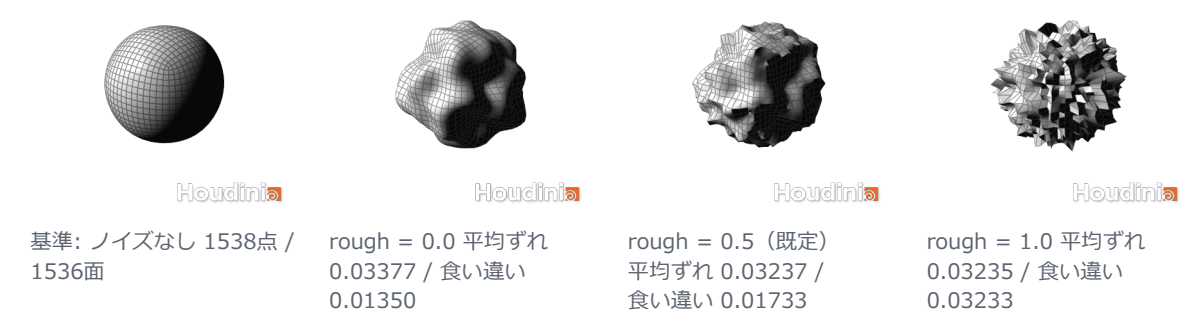
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
rough_00	mountain::2.0	elementsize = 0.25 height = 0.35 rough = 0.0
rough_05	mountain::2.0	elementsize = 0.25 height = 0.35
rough_10	mountain::2.0	elementsize = 0.25 height = 0.35 rough = 1.0
oct_1	mountain::2.0	elementsize = 0.25 height = 0.35 oct = 1
oct_3	mountain::2.0	elementsize = 0.25 height = 0.35 oct = 3
oct_8	mountain::2.0	elementsize = 0.25 height = 0.35
lac_15	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 1.5
lac_20	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 2.0
lac_40	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 4.0

ノード	タイプ	デフォルトから変更したパラメータ
fractal_hmFT	mountain::2.0	elementsiz = 0.25 height = 0.35
fractal_fBm	mountain::2.0	elementsiz = 0.25 fractal = fBm height = 0.35
fractal_mfT	mountain::2.0	elementsiz = 0.25 fractal = mfT height = 0.35
fractal_none	mountain::2.0	elementsiz = 0.25 fractal = none height = 0.35

rough（粗さ） — 振幅は変えず、細かさだけを変える

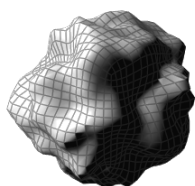
固定条件はすべての変種で共通: subdivide 4回（1538点）、height 0.35、elementsiz 0.25。rough だけを動かしている。平均のずれはほぼ変わらないのに、見た目はなだらかな丘から棘だらけの形まで変わる。



rough	平均のずれ	最大のずれ	隣との食い違い
0.0	0.03377	0.14799	0.01350
0.5（既定）	0.03237	0.13153	0.01733
1.0	0.03235	0.13588	0.03233

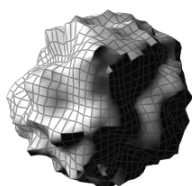
oct（オクタブ数） — 3 以上で効かなくなる

オクタブとは、細かさの違うノイズを重ねる層のこと。oct はその層の数。1 から 3 では効くが、3 から 8 ではほぼ変わらない（0.01732 → 0.01733）。点数が足りず、細かい層を表現できていないため。



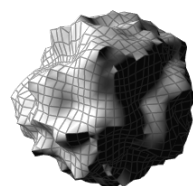
Houdini

oct = 1 食い違い 0.01519



Houdini

oct = 3 食い違い 0.01732



Houdini

oct = 8 (既定) 食い違い 0.01733

oct	平均のずれ	最大のずれ	隣との食い違い
1	0.03231	0.13417	0.01519
3	0.03250	0.13274	0.01732
8 (既定)	0.03237	0.13153	0.01733

予測の検証 — oct が効かないのは点数のせいかな

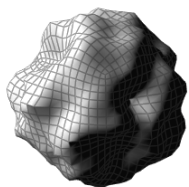
「oct が 3 以上で効かないのは点数が足りないからだ」という説明が正しいなら、点数を増やせば oct=8 が oct=3 を上回るはず。実際にそうだった。386点では oct=8 が oct=3 を下回っているが、6146点では上回っている。

注意: 食い違いの絶対値は点数が増えると小さくなる（隣同士の距離が縮まるので差も縮む）。この表は各行の中で比較だけが有効で、行をまたいで比べてはいけない。

subdivide	点数	oct = 1	oct = 3	oct = 8	8 と 3 の差
3回	386	0.02692	0.02868	0.02856	-0.00012
4回	1538	0.01519	0.01732	0.01733	+0.00001
5回	6146	0.00782	0.00941	0.00963	+0.00022

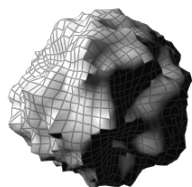
lac（ラキュナリティ） — オクターブ間の周波数の開き

重ねる層ごとに、どれだけ細かさを飛ばすかの倍率。既定は 1.841。大きくすると層の間隔が開いて細かい成分が強くなる。ただし lac = 1.5 では平均のずれも下がっており、振幅にも影響が出ている。



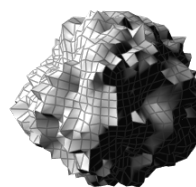
Houdini

lac = 1.5 平均 0.02625 / 食い違い 0.01259



Houdini

lac = 2.0 平均 0.03160 / 食い違い 0.01734



Houdini

lac = 4.0 平均 0.03169 / 食い違い 0.02218

lac	平均のずれ	最大のずれ	隣との食い違い
1.5	0.02625	0.13925	0.01259
2.0	0.03160	0.13528	0.01734
4.0	0.03169	0.13334	0.02218

fractal（フラクタルの種類）－ 差は小さい

層の重ね方の方式。4種類あるが、今回の条件では数値の差が小さかった。Terrain だけやや滑らか。既定は Hybrid Terrain。



Houdini

Hybrid Terrain（既定）
食い違い 0.01733



Houdini

Standard (fBm) 食い違い
0.01786



Houdini

Terrain 食い違い 0.01481



Houdini

None 食い違い 0.01350

fractal	平均のずれ	最大のずれ	隣との食い違い
Hybrid Terrain（既定）	0.03237	0.13153	0.01733
Standard (fBm)	0.03183	0.13207	0.01786
Terrain	0.03288	0.13504	0.01481
None	0.03377	0.14799	0.01350

わかったこと

- ・パラメータ名は rough / oct / lac。roughness や octaves という名前は存在せず、そのまま書くと何も設定されない。既定値は rough = 0.5、oct = 8、lac = 1.841、fractal = Hybrid Terrain、basis = Simplex。使う前に実際の名前を確認する必要がある。
- ・003で使った「基準形からのずれ」だけでは、これらのパラメータはほとんど区別できなかった。平均のずれはどれも 0.032～0.034 の範囲に収まる。振幅を決めているのは height と elementsize であり、今回のパラメータは振幅に効いていない。
- ・そこで「隣り合う点どうして、ずれの向きと量がどれだけ食い違うか」を測る指標を足した。なだらかな起伏なら隣の点も同じように動くので値は小さく、細かい凹凸なら大きくなる。基準形は原点中心なので、中心からの距離の差を符号付きのずれとして使っている（外に出たら正、へこんだら負）。これで完全に分離できた。
- ・rough が細かさに最も強く効く。0.0 から 1.0 で食い違いが 0.01350 から 0.03233 へ約2.4倍になる一方、平均のずれは 0.03377 から 0.03235 でほぼ変わらない。つまり rough は、同じ振幅のまま、なだらかな丘から棘だらけの形まで性質だけを変えるパラメータ。
- ・rough = 0.0 の数値は fractal = None と完全に一致した（平均 0.03377、最大 0.14799、食い違い 0.01350）。rough = 0 では追加の層が何も足さないの、層を重ねない単一ノイズに退化している。別の経路で同じ結果に到達したので、この解釈は確かだと言える。
- ・oct は 1 から 3 では効くが、3 から 8 ではほぼ変わらない（0.01732 → 0.01733、差は 0.00001）。点数が足りず細かい層を表現できていないため。この説明を予測に変えて検証したところ、点数を増やす

と oct = 8 が oct = 3 を上回り始めた（386点では -0.00012、1538点で +0.00001、6146点で +0.00022）。予測どおりだったので、この解釈も確かだと言える。

- ・実用的には、oct を上げるだけでは無駄になる。点数が足りていなければ、上げた層は形に現れない。003の「点数は解像度を決める」という結論と同じことが、別の角度から確認できた。

- ・lac も細かさに効く（1.5 → 4.0 で 0.01259 → 0.02218）。ただし lac = 1.5 では平均のずれも 0.02625 に下がっており、振幅にも影響している。rough のようにきれいに性質だけを変えるパラメータではない。

- ・fractal の4種類は、今回の条件では差が小さかった（食い違い 0.01350~0.01786）。地形向け的方式が揃っているので、平面の地形（実験005の予定）ではもっと差が出る可能性がある。

- ・指標の注意点。食い違いの絶対値は点数が増えると小さくなる。隣同士の距離が縮まれば、その間のずれの差も縮むため。したがって同じ点数の中でだけ比較できる。点数をまたいだ比較には、辺の長さで割るなどの正規化が必要になる。

- ・basis（ノイズの基礎となる種類）は14種類あった。既定は Simplex。Worley Cellular のようなセル状のものも含まれており、見た目が大きく変わるはずなので、別の実験にする価値がある。

次にやること

- ・basis を14種類すべて描き出して並べる。セル状ノイズは岩や結晶の見た目に直結するので、実用価値が高い
- ・grid から地形を作り、平面では fractal の種類の差が出るかを確かめる（実験005）
- ・食い違いの指標を辺の長さで正規化し、点数をまたいだ比較ができるようにする

basis（ノイズの種類） 14通りで、形はどこまで変わるか

2026-09-13 | Houdini 21.0.700 | /obj/noise_basis | ノード 16 / 接続 15

何を作ったか

004で basis に14種類あることが分かったので、同じ条件で全部描き出した。height 0.35、elementsize 0.25、subdivide 4回（1538点）で固定し、basis だけを変えている。

結果は他のパラメータと性質がまったく違った。rough は振幅を変えずに形の性質だけを変えたが、basis は振幅も性質も両方変える。同じ height を指定しても、種類によって実際のずれが4.5倍違う。

測る指標には004で作った「隣との食い違い」に加えて、外に出た点の割合を足した。ノイズが外向きに押すのか内向きに削るのかで、使いどころが変わるため。

ノードの接続



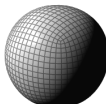
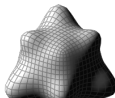
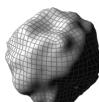
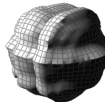
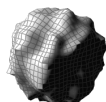
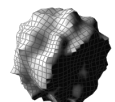
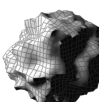
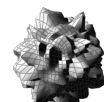
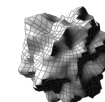
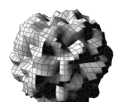
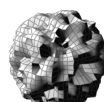
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
basis_sine	mountain::2.0	basis = sine elementsize = 0.25 height = 0.35
basis_perlin	mountain::2.0	basis = perlin elementsize = 0.25 height = 0.35
basis_pperlin	mountain::2.0	basis = pperlin elementsize = 0.25 height = 0.35
basis_simplex	mountain::2.0	elementsize = 0.25 height = 0.35
basis_sparse	mountain::2.0	basis = sparse elementsize = 0.25 height = 0.35
basis_flow	mountain::2.0	basis = flow elementsize = 0.25 height = 0.35
basis_pflow	mountain::2.0	basis = pflow elementsize = 0.25 height = 0.35
basis_worleyFA	mountain::2.0	basis = worleyFA elementsize = 0.25 height = 0.35
basis_worleyFB	mountain::2.0	basis = worleyFB elementsize = 0.25 height = 0.35

ノード	タイプ	デフォルトから変更したパラメータ
basis_mworleyFA	mountain::2.0	basis = mworleyFA elementsiz = 0.25 height = 0.35
basis_mworleyFB	mountain::2.0	basis = mworleyFB elementsiz = 0.25 height = 0.35
basis_cworleyFA	mountain::2.0	basis = cworleyFA elementsiz = 0.25 height = 0.35
basis_cworleyFB	mountain::2.0	basis = cworleyFB elementsiz = 0.25 height = 0.35
basis_alligator	mountain::2.0	basis = alligator elementsiz = 0.25 height = 0.35

14種類すべて（細かさの小さい順）

すべて同じカメラ・同じ設定。左上のなめらかなものから、右下の硬く割れたものへ並んでいる。

					
Houdini	Houdini	Houdini	Houdini	Houdini	
基準: ノイズなし 1538点	Sinusoid 平均 0.14454 / 細かさ 0.00444	Sparse Convolution 平均 0.01985 / 細かさ 0.00965	Periodic Perlin Flow 平均 0.01551 / 細かさ 0.01005	Periodic Perlin 平均 0.01662 / 細かさ 0.01062	
					
Houdini	Houdini	Houdini	Houdini	Houdini	
Perlin 平均 0.02426 / 細かさ 0.01268	Perlin Flow 平均 0.02639 / 細かさ 0.01441	Simplex (既定) 平均 0.03237 / 細かさ 0.01733	Worley F2-F1 平均 0.14144 / 細かさ 0.01831	Worley F1 平均 0.08017 / 細かさ 0.02065	
					
Houdini	Houdini	Houdini	Houdini	Houdini	
Alligator 平均 0.12904 / 細かさ 0.02747	Manhattan F1 平均 0.08103 / 細かさ 0.02996	Chebyshev F2-F1 平均 0.11841 / 細かさ 0.03104	Chebyshev F1 平均 0.09937 / 細かさ 0.03193	Manhattan F2-F1 平均 0.11505 / 細かさ 0.03323	
basis	表示名	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
sine	Sinusoid	0.14454	0.15748	0.00444	0.0%

basis	表示名	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
sparse	Sparse Convolution	0.01985	0.08378	0.00965	32.9%
pflow	Periodic Perlin Flow	0.01551	0.05039	0.01005	51.2%
pperlin	Periodic Perlin	0.01662	0.05705	0.01062	58.8%
perlin	Perlin	0.02426	0.09921	0.01268	40.9%
flow	Perlin Flow	0.02639	0.10496	0.01441	49.9%
simplex	Simplex (既定)	0.03237	0.13153	0.01733	35.8%
worleyFB	Worley Cellular F2-F1	0.14144	0.17436	0.01831	0.0%
worleyFA	Worley Cellular F1	0.08017	0.17328	0.02065	5.4%
alligator	Alligator	0.12904	0.17456	0.02747	2.3%
mworleyFA	Manhattan Cellular F1	0.08103	0.16473	0.02996	91.1%
cworleyFB	Chebyshev Cellular F2-F1	0.11841	0.17439	0.03104	1.4%
cworleyFA	Chebyshev Cellular F1	0.09937	0.16540	0.03193	94.4%
mworleyFB	Manhattan Cellular F2-F1	0.11505	0.17052	0.03323	2.5%

わかったこと

- basis は振幅と性質の両方を変える。rough

が振幅を保ったまま性質だけを変えたのとは性格が違う。同じ height 0.35 でも、平均のずれは Periodic Perlin Flow の 0.01551 から Sinusoid の 0.14454 まで約9.3倍の開きがある。basis を変えたら height も再調整が必要。

- Sinusoid は例外的な挙動をする。平均のずれが 0.14454 と最大級に大きいのに、隣との食い違いは 0.00444 で全種類中もっとも小さい。外向きの点は 0.0%、つまり全部の点が内側に動いている。画を見ると、丸い4つの膨らみを持つ滑らかな形になっていた。規則的な波なので、細かい凹凸ではなく大きなうねりを作る。地形の大きな起伏には使えるが、ディテールには使えない。

- Perlin 系 (perlin / pperlin / flow / pflow) は外向きの点が 40~59% で、外と内にほぼ均等に動く。平均のずれも 0.0155~0.0264 と控えめ。自然な揺らぎを足したいときの素直な選択肢。

- 既定の Simplex は Perlin 系より振幅も細かさも大きい (平均 0.03237、細かさ 0.01733) が、セル系ほど極端ではない。中間的な位置にある。

- セル系 (Worley / Manhattan / Chebyshev / Alligator) は平均のずれが 0.08~0.14 と大きく、隣との食い違いも上位を占める。岩、割れ、結晶のような硬い見た目はここから作る。

- セル系の中で、外向きの点の割合が極端に分かれた。Manhattan F1 が 91.1%、Chebyshev F1 が 94.4% で、ほぼ全部の点が外に出る。一方 Worley F1 は 5.4%、Alligator は 2.3%、Chebyshev F2-F1 は 1.4% で、ほぼ全部が内側に削られる。

- ・画で確認すると理由が見えた。Worley F1

は、細胞の中心から遠いところが深く削られた侵食岩のような形になる。Manhattan F1 は距離の測り方が軸に沿っているため、軸に平行な面を持つ角ばった塊が外に突き出す形になった。同じ「F1」でも距離の定義が違うと、削る方向と形がまったく変わる。

・ F1 と F2-F1 の違いも外向き率に出る。Manhattan は F1 が 91.1% (外向き) なのに F2-F1 は 2.5% (内向き)。Chebyshev も F1 が 94.4%、F2-F1 が 1.4% で同じ傾向。F2-F1 は細胞の境界線を強調する計算なので、境界以外が下がる = 内向きになる。

・最大のずれはセル系でほぼ頭打ちになっている (0.164~0.175)。height 0.35 に対して 0.5 倍弱で止まっており、何らかの上限が効いている可能性がある。この確認は別の実験にする。

・外向きの点の割合という指標は安く作れて情報量が多かった。ノイズが押すのか削るのかは、実際に作りたいもの (突起か窪みか) に直結する。

次にやること

- ・セル系で最大のずれが 0.175 付近で止まる理由を確かめる。height を上げても止まるのかを測る
- ・grid の平面で同じ14種類を並べる。地形として見たときの向き不向きは球体とは違うはず
- ・外向き率を、突起を作りたいとき／窪みを作りたいときの選択指針としてまとめる

height は純粋な倍率か。gain と bias は何に効くか

2026-09-13 | Houdini 21.0.700 | /obj/height_shape | ノード 10 / 接続 9

何を作ったか

005でセル系の最大のずれが 0.164~0.175
に揃っていたので、何らかの上限が効いているのではないかと書いた。それを確かめる実験。height を 0.0875 から 1.4 まで16倍の範囲で変え、最大のずれが比例するかを見る。

結論を先に書くと、005の見立ては誤りだった。上限は存在しない。そして高い height で比率が崩れたのは Houdini の制限ではなく、私が使っている指標が壊れていたためだった。

あわせて未調査だった gain / bias / clipmin / clipmax も測った。bias は実用上かなり重要なパラメータだと分かった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
h_simplex_035	mountain::2.0	elementsize = 0.25 height = 0.35
h_worleyFA_035	mountain::2.0	basis = worleyFA elementsize = 0.25 height = 0.35
gain_025	mountain::2.0	dogain = 1 elementsize = 0.25 gain = 0.25 height = 0.35
gain_05	mountain::2.0	dogain = 1 elementsize = 0.25 height = 0.35
gain_075	mountain::2.0	dogain = 1 elementsize = 0.25 gain = 0.75 height = 0.35
bias_025	mountain::2.0	bias = 0.25 dobias = 1 elementsize = 0.25 height = 0.35
bias_05	mountain::2.0	dobias = 1 elementsize = 0.25 height = 0.35
bias_075	mountain::2.0	bias = 0.75 dobias = 1 elementsize = 0.25 height = 0.35

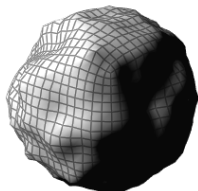
height を変えたときの最大のずれ

固定条件: subdivide 4回 (1538点)、elementsize 0.25。「最大 / height」が一定なら、height は単なる倍率ということになる。

basis	height	平均のずれ	最大のずれ	最大 / height	隣との食い違い
simplex	0.0875	0.00809	0.03289	0.37594	0.00433
simplex	0.1750	0.01619	0.06578	0.37590	0.00867
simplex	0.3500	0.03237	0.13153	0.37579	0.01733
simplex	0.7000	0.06473	0.26273	0.37532	0.03466
simplex	1.4000	0.12822	0.43122	0.30801	0.06868
worleyFA	0.0875	0.02005	0.04332	0.49513	0.00516
worleyFA	0.1750	0.04009	0.08665	0.49512	0.01033
worleyFA	0.3500	0.08017	0.17328	0.49508	0.02065
worleyFA	0.7000	0.16025	0.34636	0.49480	0.04125
worleyFA	1.4000	0.26687	0.42980	0.30700	0.07338

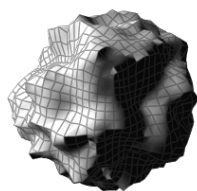
gain — 振幅の倍率。外向きと内向きの比率は変えない

gain は dogain を有効にしないと効かない（既定は無効）。gain = 0.5 のときの数値は、dogain を無効にした既定状態と完全に一致した。つまり 0.5 が中立値。



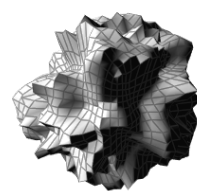
Houdini

gain 0.25 平均 0.01494 / 外向き 35.8%



Houdini

gain 0.50 (中立) 平均 0.03237 / 外向き 35.8%



Houdini

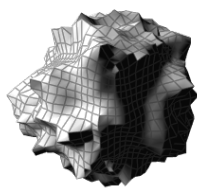
gain 0.75 平均 0.05519 / 外向き 35.8%

gain	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
0.25	0.01494	0.07691	0.00816	35.8%
0.50 (中立)	0.03237	0.13153	0.01733	35.8%
0.75	0.05519	0.16402	0.02870	35.8%

bias — 外向きと内向きの比率を直接変える

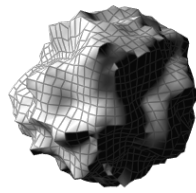
bias も dobias を有効にしないと効かない。0.5 が中立。0.25 にするとほぼ全部が内側に削られ、0.75 にするとほぼ全部が外側に膨らむ。005では basis

を選ぶことでしか得られなかった「突起か窪みか」の制御が、1つのパラメータで連続的にできる。



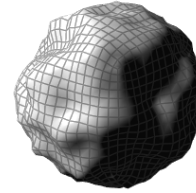
Houdini

bias 0.25 外向き 0.9% =
ほぼ全部が窪み



Houdini

bias 0.50 (中立) 外向き 35.8%



Houdini

bias 0.75 外向き 98.7% =
ほぼ全部が膨らみ

bias	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
0.25	0.09756	0.16935	0.01614	0.9%
0.50 (中立)	0.03237	0.13153	0.01733	35.8%
0.75	0.07644	0.14054	0.01156	98.7%

わかったこと

・ height は純粋な倍率だった。「最大 / height」は 0.0875 から 0.7

まで8倍の範囲で小数点以下4桁まで一定 (simplex は 0.3759、worleyFA は 0.4951)。height を2倍にすれば、ずれもきっちり2倍になる。

・ 005で疑った上限は存在しなかった。セル系の最大のずれが揃って見えたのは、セル系ノイズのピーク値の分布が互いに似ているため。実際 005 のセル系7種類の「最大 / height」を計算すると 0.47~0.50 の範囲に収まる。一方 simplex は 0.376

で明確に違う。上限ではなく、ノイズの種類ごとの固有の比率だった。

・ ただし height = 1.4 では比率が両方とも 0.307 に落ちた。これは Houdini の制限ではなく、私の指標が壊れているため。基準形の半径は約 0.42 (subdivide 4回のバウンディングボックスの一辺 0.84192 の半分) で、そのとき測れた最大のずれ 0.4312 / 0.4298 はこれとほぼ一致している。中心を貫通するほど強く押し込まれると、「中心からの距離の差」という測り方では値が頭打ちになる。

・ つまりこの指標には適用範囲がある。ずれが対象の半径より十分小さいときにしか使えない。今回でいえば height 0.7 まで。005の観察を「上限がある」と解釈しかけたのは、この限界を意識していなかったため。指標を作ったら、それが壊れる条件も押さえておく必要がある。

・ clipmin / clipmax の既定は 0.0 / 1.0、dogain / dobias は既定で無効。つまり既定状態ではクリップもゲインもバイアスも効いていない。005までの測定に余計な処理は入っていなかったことが確認できた。

・ gain は振幅の倍率として働く。0.25 / 0.5 / 0.75 で平均のずれが 0.01494 / 0.03237 / 0.05519 と増える。一方で外向きの点の割合は 35.8%

のまま完全に動かない。形の偏りには触れず、強さだけを変えるパラメータ。

・ gain = 0.5 の数値は、dogain を無効にした既定状態と完全に一致した (平均 0.03237、最大 0.13153、細かさ 0.01733)。0.5 が中立値だと確認できた。

・ bias は外向きと内向きの比率を直接変える。0.25 で外向き 0.9% (ほぼ全部が内側に削られる)、0.5 で 35.8%、0.75 で 98.7% (ほぼ全部が外側に膨らむ)。画で見ると、0.25 は深くえぐられて鋭い稜線が残る侵食的な形、0.75 は膨らんで浅いへこみが残る塊になる。

- ・ただし bias を 0.5 から動かすと平均のずれも増える (0.03237 → 0.09756 / 0.07644)。分布が片側に寄ると、ゼロから離れた点が増えるため。純粋に形の偏りだけを変えるパラメータではなく、振幅も一緒に動く。

- ・ここまでで mountain のパラメータ地図がほぼ埋まった。振幅を決めるのは height (純粋な倍率) と gain (倍率)。細かさを決めるのは rough (最も強い)、lac、oct (点数が足りている範囲で)。外向きか内向きかを決めるのは bias (連続的) と basis (種類で決まる)。そして何を表現できるかの上限は点数が決める。

次にやること

- ・grid の平面で同じ測定をする。球体では中心からの距離で測れたが、平面では別の基準が必要になるので、指標の作り直しも兼ねる

- ・指標を辺の長さで正規化し、点数をまたいだ比較ができるようにする

- ・パラメータ調査はここで区切り、scatter と copy to points に進む

平面で地形を作り、指標を作り直す

2026-09-13 | Houdini 21.0.700 | /obj/grid_terrain | ノード 12 / 接続 8

何を作ったか

これまでの指標は「中心からの距離の差」で、球体でしか使えず、点数をまたいだ比較もできなかった（006でその限界にぶつかった）。平面なら高さの差をそのまま測れる。さらに点の間隔で割れば勾配（傾き）になるので、これなら解像度によらない量になるはずだと考えた。

結果として、その期待は外れた。勾配は解像度を上げるほど増え続ける。ただしこれは失敗ではなく、狙っていたものより役に立つ道具になった。勾配が増えなくなる点が「その形を表現するのに十分な解像度」を教えてくれる。

あわせて、004で「平面ならもっと差が出るかもしれない」と書いた fractal の種類も確かめた。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
grid_20	grid	cols = 20 rows = 20
mt_20	mountain::2.0	elementsize = 2.0
grid_40	grid	cols = 40 rows = 40
mt_40	mountain::2.0	elementsize = 2.0
grid_80	grid	cols = 80 rows = 80
mt_80	mountain::2.0	elementsize = 2.0
grid_160	grid	cols = 160 rows = 160
mt_160	mountain::2.0	elementsize = 2.0
frac_none	mountain::2.0	elementsize = 2.0 fractal = none
frac_fBm	mountain::2.0	elementsize = 2.0 fractal = fBm
frac_mfT	mountain::2.0	elementsize = 2.0 fractal = mfT
frac_hmfT	mountain::2.0	elementsize = 2.0

解像度を変える（elementsize 2.0、height 1.0 固定）

10×10 の grid の分割数を 20 から 160 まで上げた。点の間隔はそれに応じて 0.53 から 0.06 まで細くなる。

分割数	点数	点の間隔	平均の高さ	最大の高さ	平均勾配
20	400	0.5263	0.07919	0.29407	0.13595

分割数	点数	点の間隔	平均の高さ	最大の高さ	平均勾配
40	1600	0.2564	0.08187	0.32680	0.16634
80	6400	0.1266	0.08183	0.32995	0.18745
160	25600	0.0629	0.08196	0.33513	0.20430

オクターブ数ごとに、勾配がどこで止まるか

各行が1つの解像度。オクターブ数を増やすほど細かい成分が入るので、それを表現しきるのに必要な解像度も上がる。数字が横に並んで止まっていれば、その解像度で足りているということ。

分割数	oct = 1	oct = 2	oct = 4	oct = 8	oct=8 の 80→160 増加率
20	0.12897	0.13673	0.13593	0.13595	—
40	0.14205	0.15845	0.16659	0.16634	—
80	0.14492	0.16561	0.18604	0.18745	—
160	0.14570	0.16752	0.19605	0.20430	+9.0%
収束したか	した (+0.5%)	ほぼした (+1.2%)	まだ (+5.4%)	まだ (+9.0%)	

fractal の4種類（分割数80）

004では球体で比べて差が小さかった（細かさ 0.01350~0.01786）。平面では勾配 0.12216 から 0.19064 まで1.6倍の開きが出た。004で立てた「平面ならもっと差が出るかもしれない」という見込みは当たっていた。



Houdini

None 勾配 0.12216 — 最も滑らか



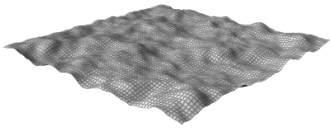
Houdini

Terrain 勾配 0.14766



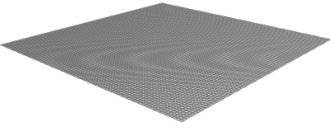
Houdini

Hybrid Terrain（既定） 勾配 0.18745



Houdini

Standard (fBm) 勾配 0.19064 — 最も粗い



Houdini

基準: ノイズなしの grid 6400点 / 6400面

fractal	平均の高さ	最大の高さ	平均勾配	上向きの点の割合
None	0.08386	0.39362	0.12216	44.4%
Terrain	0.08257	0.38162	0.14766	43.6%

fractal	平均の高さ	最大の高さ	平均勾配	上向きの点の割合
Hybrid Terrain (既定)	0.08183	0.32995	0.18745	42.3%
Standard (fBm)	0.08269	0.37818	0.19064	45.4%

わかったこと

- ・ grid SOP の既定は 10×10 のサイズ、分割は 10×10、向きは ZX 平面（水平）、種類はポリゴン。水平に置かれるので、mountain の押し出しは真上（Y方向）になる。だから高さの差をそのまま測れる。
- ・ 狙いは外れた。勾配は解像度によらない量にはならず、分割数を上げるほど増え続けた（0.13595 → 0.16634 → 0.18745 → 0.20430）。fBm 系のノイズは細かい成分を何段も持っているので、細かく刻むほど急な斜面が新たに見えてくるため。
- ・ ただしこの性質のおかげで、別のことが分かるようになった。オクターブ数ごとに勾配の伸びを見ると、oct = 1 は 80分割で収束し（80→160 で +0.5%）、oct = 2 はほぼ収束（+1.2%）、oct = 4 はまだ伸び（+5.4%）、oct = 8 もまだ伸びている（+9.0%）。つまり勾配が増えなくなったら、その解像度でその形を表現しきれているという判定に使える。
- ・ 低い解像度では余分なオクターブがまったく見えない。20分割では oct = 2 / 4 / 8 がほぼ同じ値（0.13673 / 0.13593 / 0.13595）になり、oct = 2 がわずかに最大という順序の逆転まで起きている。004で球体を使って得た結論が平面でも再現し、しかも解像度との関係まで数字で見えた。
- ・ 高さの平均は40分割以降ほぼ一定（0.0819前後）。振幅は低い解像度でもすぐ正しくなるが、細かさはずっと遅れて収束する。形が正しく見えるのに必要な解像度は、高さが合う解像度よりはるかに高いということ。点数を決めるときは高さではなく細かさを基準にする必要がある。
- ・ 最大の高さも解像度とともに増える（0.29407 → 0.33513）。これも同じ理由で、細かく刻むほどノイズの尖った部分を拾えるようになるため。005・006で使った「最大のずれ」も同じ性質を持っていたことになる。
- ・ fractal の種類は平面で明確に差が出た。滑らかな順に None（0.12216）、Terrain（0.14766）、Hybrid Terrain（0.18745）、Standard fBm（0.19064）。一方で平均の高さはどれも 0.0818~0.0839 でほぼ同じ。fractal は振幅ではなく細かさを変えるパラメータだと確定した。
- ・ None が最も滑らかなのは筋が通っている。層を重ねない単一のノイズなので、細かい成分が入らない。004で「rough = 0 は fractal = None と完全に一致する」と分かったことと整合する。
- ・ カメラのフレーミングを直した。以前はバウンディングボックスを立方体とみなして距離を決めていたため、平たい地形が画面の4分の1ほどにしか写らなかった。8つの隅をカメラ空間に投影して必要な距離を解く方式に変更し、対象が画面いっぱい収まるようになった。地形を扱う以上、これは必須の修正だった。

次にやること

- ・ elementsize・オクターブ数・必要な分割数の関係を式にまとめる。「この設定ならこれだけの点数が必要」と事前に見積もれるようにする
- ・ 地形専用の heightfield ノード群と、この grid + mountain のやり方を比べる
- ・ パラメータの調査はここで区切り、scatter と copy to points に進む

scatter と copy to points — 地面に物を生やす

2026-09-13 | Houdini 21.0.700 | /obj/scatter_copy | ノード 12 / 接続 13

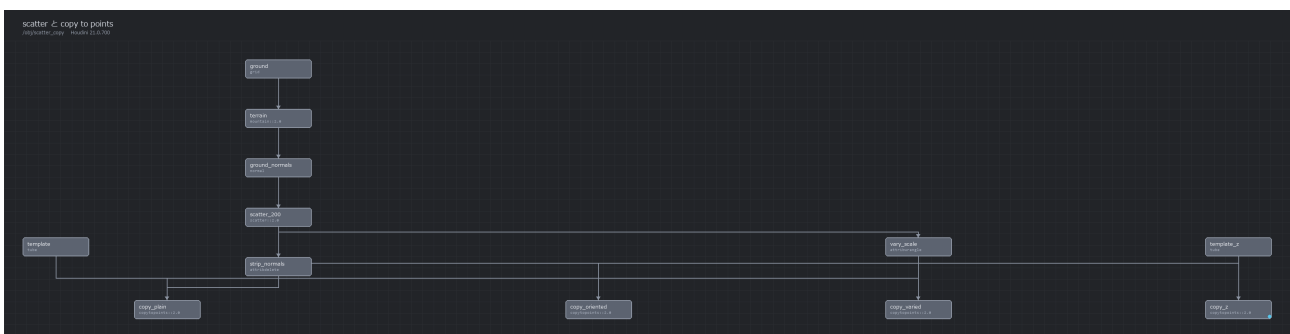
何を作ったか

面の上に点をばらまき、その点ごとにジオメトリを複製する。プロシージャルモデリングの中核パターンで、草や岩や建物を並べるのは全部これで行う。

測ったのは3つ。散布点の数は指定どおりになるか、複製後の点数はどう増えるか、そして複製が地面の傾きに沿って回るかどうか。

3つめで重要な落とし穴が見つかった。複製は確かに回るのだが、回り方が想像と違う。数字で確定させた。

ノードの接続



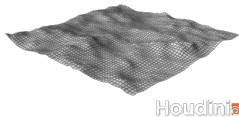
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
ground	grid	cols = 60 rows = 60
terrain	mountain::2.0	elementsizе = 3.0 height = 1.2
ground_normals	normal	(変更なし)
scatter_200	scatter::2.0	npts = 200
template	tube	cap = 1 cols = 10 height = 0.9 rad1 = 0.0 rad2 = 0.18 type = 1
strip_normals	attribdelete	ptdel = N
copy_plain	copytopoints::2.0	(変更なし)
copy_oriented	copytopoints::2.0	(変更なし)
vary_scale	attribwrangle	snippet = @pscale = fit01(rand(@ptnum * 7.3), 0.35, 1.4);
copy_varied	copytopoints::2.0	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
template_z	tube	cap = 1 cols = 10 height = 0.9 orient = 2 rad1 = 0.0 rad2 = 0.18 type = 1
copy_z	copytopoints::2.0	(変更なし)

複製の向きと大きさ

地面は 10×10 の grid（60分割）に mountain をかけたもの。散布点は200個、テンプレートは20点の円錐。3つの条件で複製した。



地面 grid 60分割 + mountain



法線なしの点に複製
向きのばらつき 0.00度



法線ありの点に複製 + pscale
向きのばらつき 10.84度



テンプレートをZ軸向きにした場合 N と完全に一致する

条件	複製後の点数	複製数	予想点数	向きのばらつき（平均 / 最大）	大きさの幅
法線なし	4000	200	4000	0.00度 / 0.00度	0.45 固定
法線あり	4000	200	4000	10.84度 / 31.69度	0.45 固定
法線あり + pscale	4000	200	4000	10.84度 / 31.69度	0.16 ~ 0.63

N はテンプレートのどの軸に対応するのか

複製が地面に沿って回ってはいるのに、円錐が横倒しに見えた。そこで、各複製の軸と、その複製が乗っている点の N との角度を全200個について測った。結果はばらつきゼロで、完全に一方の答えを示した。

テンプレートの向き	N との角度（平均）	最小	最大
Y軸（tube の既定）	90.00度	90.00度	90.00度
Z軸	0.00度	0.00度	0.00度

実際の Houdini の画面

自動生成の接続図と違い、これは起動中の Houdini を外から操作して撮った本物の画面。自分の Houdini で同じものを開いたときに何が見えるかがそのまま分かる。パラメータ側に写っている「Transform Using Target Point Orientations」が、この実験で測った「N で回転するかどうか」の正体。



ネットワークエディタ。scatter_200 から4つの Copy to Points に分岐している



copy_oriented のパラメータと、その結果のビューポート

scatter の点数は指定どおりか

指定した数	実際の点数	一致
50	50	はい
200	200	はい
800	800	はい

わかったこと

- ・ scatter の npts は正確に守られる。50 / 200 / 800 を指定して、実際の点数も 50 / 200 / 800 になった。おおよその数ではなく、指定した数ちょうどが出る。
- ・ scatter の出力は P だけでなく N も持っている。元の面の法線が引き継がれるため。この N があるかないかで、次の copy to points の挙動が変わる。
- ・ copy to points の点数は掛け算そのまま。散布点200個 × テンプレート20点 = 4000点。予想と実測が完全に一致した。テンプレートが重いと一気に破綻するので、散布数とテンプレートの点数は掛け算で効くと覚えておく。

- ・ N を持たない点に複製すると、複製は一切回転しない。向きのばらつきは平均も最大も 0.00度。200個すべてが同じ向きに立つ。

- ・ N を持つ点に複製すると、点ごとに回転する。向きのばらつきは平均 10.84度、最大 31.69度で、これは地面の傾きのばらつきそのもの。

- ・ ただし N はテンプレートの Y 軸ではなく Z 軸に対応している。tube の既定 (Y軸向き) のまま複製すると、複製の軸と N の角度が全200個すべて正確に 90.00度、つまり完全に横倒しになる。テンプレートを Z軸向きに作り直すと全200個すべて 0.00度で完全に一致した。ばらつきがゼロなので、これは偶然ではなく仕様。

- ・ tube SOP の orient の既定は Y軸。そのため何も考えずに tube を複製すると必ず横倒しになる。地面に立てたいなら、テンプレートを Z軸向きに作るか、あとで回転させる必要がある。

- ・ pscale は大きさだけを変える。向きのばらつきは 10.84度 / 31.69度 のまま完全に同じで、大きさが 0.45 固定から 0.16~0.63 に広がった。向きと大きさは独立して制御できる。

- ・ tube SOP の type の既定は Primitive で、1点 / 1面の「本物の円柱プリミティブ」になる。ポリゴンとして扱いたいなら Polygon に変える必要がある。これに気づかず、最初は1点のテンプレートを200個複製していた (200点になるだけで何も見えない) 。

- ・ 測り方の失敗も1つ記録しておく。最初は「重心から最も遠い点」で複製の向きを測ろうとしたが、円錐の底面リングの点がすべて等距離のため、選ばれる点が複製ごとにばらついて 21.44度 という無意味な値が出た。固定の点番号を使う方式に変えたら 0.00度 になった。等距離の点が並ぶ形では「最も遠い点」は一意に決まらない。

- ・ 実用の型はこうなる。地面 → normal → scatter → copy to points。テンプレートは Z軸向きに作る。大きさを変えたいなら散布点に pscale を持たせる。この4ノードで「地面に物を生やす」が完成する。

次にやること

- ・ 複製の向きに、傾きだけでなくランダムな回転を混ぜる (up ベクトルの扱い)
- ・ density や面積のアトリビュートで散布密度に偏りをつける
- ・ for-each で複製ごとに違う処理をかける (実験010の予定)

attribute wrangle で VEX を書く ー 何が何回動くのか

2026-09-13 | Houdini 21.0.700 | /obj/vex_intro | ノード 2 / 接続 1

何を作ったか

VEX は「ノードを並べる代わりに数行書く」ための言語。ただし初心者が最初につまずくのは文法ではなく、そのコードが何に対して何回走るのかという点。それを実測で確かめる。

あわせて「VEXは速い」という話を数字にしようとしたが、ここで測り方を二度も間違えた。最初の方法では計算量を100倍にしても時間が変わらず、公式の性能計測機能を使っても同じだった。三度目でようやく実際の計算を捉えられた。その経緯も残す。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
grid	grid	cols = 20 rows = 20
wave	attribwrangle	snippet = @P.y = sin(@P.x * 2.0) * 0.5;

実行対象を変えると、作られるアトリビュートの種類が変わる

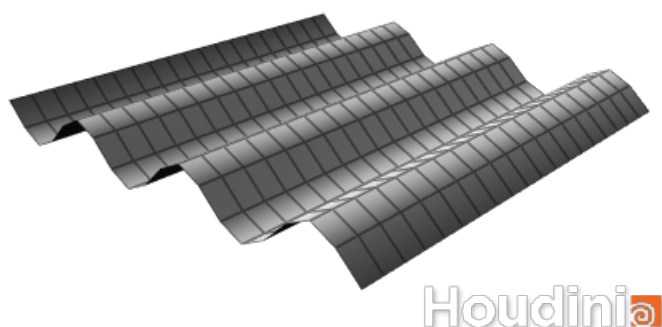
20×20 の grid（ポイント400 / プリミティブ361 / バーテックス1444）に対して、同じコード @marker = 1.0; を実行対象だけ変えて走らせた。

実行対象（class）	作られたアトリビュートの種類	対象の要素数
Detail（only once）	ディテール	1
Primitives	プリミティブ	361
Points（既定）	ポイント	400

実行対象 (class)	作られたアトリビュートの種類	対象の要素数
Vertices	バーテックス	1444
Numbers	(作られない)	—

予測してから測る — $@P.y = \sin(@P.x * 2.0) * 0.5$

grid は 10×10 で原点中心なので x は -5 ~ +5。sin の値は -1 ~ +1 の範囲を取るので、y はちょうど -0.5 ~ +0.5 に収まるはず。



VEX 1行で作った波 $@P.y = \sin(@P.x * 2.0) * 0.5;$

	値
予測した範囲	-0.5 ~ +0.5
実測した範囲	-0.499983 ~ +0.499983
差	0.000017

点数と処理時間（毎回新しいノードを作り、5回測った中央値）

1秒あたりの点数が点数とともに増えているのは、ノードを作る固定の費用が大きいため。点が増えるほどその費用が薄まる。

分割数	点数	中央値（ミリ秒）	1秒あたりの点数
200	40,000	1.487	2,690万
400	160,000	2.005	7,979万
800	640,000	2.472	2億5,891万
1600	2,560,000	4.580	5億5,895万

その計測は本物か — 同じ点数で計算量だけ増やす（64万点）

計算量を約100倍にしたときに時間が増えなければ、計測が実際の計算を捉えていないということ。結果の最大yが変わっていることも、確かに別の計算が走った証拠になる。

処理	中央値（ミリ秒）	軽いものとの比	結果の最大 y
軽い (sin × cos を1回)	2.48	1.0倍	0.49999

処理	中央値（ミリ秒）	軽いものとの比	結果の最大 y
重い（100回の繰り返し）	43.91	17.7倍	0.32169

わかったこと

- ・ attribute wrangle の「実行対象（class）」は5種類ある。Detail (only once) / Primitives / Points / Vertices / Numbers で、既定は Points。
- ・ 実行対象を変えると、作られるアトリビュートの種類がそれに従って変わる。同じ @marker = 1.0; というコードでも、Points なら点に、Primitives なら面に、Vertices なら頂点に、Detail ならジオメトリ全体に1つだけ作られる。コードは同じでも結果の置き場所が変わるというのが、最初に押さえるべき点。
- ・ Numbers だけはアトリビュートが作られなかった。これはジオメトリの要素ではなく、指定した個数の番号に対して走るモードなので、値を書き込む先がないため。
- ・ 予測を立ててから測る形にした。sin の値域は -1 ~ +1 なので、0.5 倍すれば y は -0.5 ~ +0.5 に収まるはず。実測は -0.499983 ~ +0.499983 だった。
- ・ この 0.000017 のずれも説明できる。grid は20分割なので点の間隔は $10/19 = 0.526316$ 。sin(2x) が最大になるのは $x = \pi/4 \approx 0.785398$ のときだが、最も近い格子点は $x = 0.789474$ で、わずかに外れている。この点での値を計算すると 0.4999834 になり、実測値と小数点以下6桁まで一致する。誤差ではなく、格子がピークをまたいでいるだけだった。
- ・ rand(@ptnum) は完全に再現する。同じコードを別のノードで2回走らせて、全400点が小数点以下9桁まで一致した。乱数といっても点番号から決まる固定の値なので、実行するたびに変わることはない。
- ・ その分布も妥当だった。0~2 の範囲に散らす設定で平均 1.03046（理論値 1.0）、範囲は 0.00103 ~ 1.99050。400点しかないので 3% ほどのずれは想定内。
- ・ 処理時間の計測は二度失敗した。一度目は cook(force=True) の前後を time.perf_counter で挟む方法。これだと計算量を100倍にしても時間が 0.9 倍にしかならなかった。ジオメトリを要求した時点で初めて計算されるため、計測区間の外で処理が走っていた。
- ・ 二度目は Houdini の性能計測機能（hou.perfMon）を使ったが、軽い処理も重い処理も同じ 0.008000 秒を返した。同じノードを焼き直す限り、キャッシュが効いて実際には再計算されないらしい。
- ・ 三度目にようやく成功した。毎回新しいノードを作って初回のクックを測る。キャッシュが存在しないので必ず計算が走る。この方法だと計算量100倍の処理が 17.7倍 の時間になり、結果の最大yも 0.49999 から 0.32169 に変わった。時間と結果の両方が動いたので、今度は本物。
- ・ 得られた数字。64万点に sin × cos を1回かける処理が 2.47ミリ秒、256万点で 4.58ミリ秒。この2点から傾きを出すと、1点あたりの限界コストは約1.1ナノ秒（毎秒約9億点）、固定の費用が約1.8ミリ秒。
- ・ つまり「VEXが速い」効果は点数が多いときにしか出ない。4万点では1.49ミリ秒かかっており、そのほとんどが固定費。少ない点数なら、どんな書き方をしても体感は変わらない。
- ・ 17.7倍という比率が100倍にならないのは、繰り返しの中身が単純で、ノードを作る固定費も含まれているため。それでも計算量に応じて時間が動くことは確認できたので、計測としては成立している。

次にやること

- ・ for-each ループと VEX の使い分け。同じことをどちらでもできる場面で、書きやすさと速さを比べる
- ・ VEX からアトリビュートを読み書きする（point() / setpointattrib()）
- ・ 測り方の失敗が3回続いたので、計測の型を作っておく。「結果が変わること」と「時間が変わること」の両方を確認するのを標準にする

for-each ループと VEX — 同じことをやって比べる

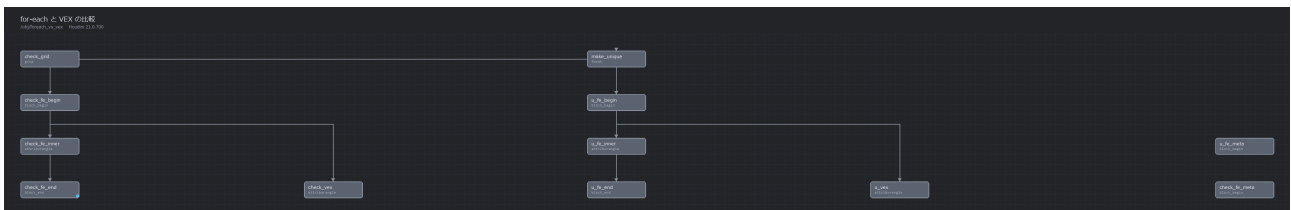
2026-09-13 | Houdini 21.0.700 | /obj/foreach_vs_vex | ノード 12 / 接続 9

何を作ったか

「まとめて処理できないときは for-each」とよく言われるが、VEX でも書ける場面でどれだけ差が出るのかは測らないと分からない。そこで、面ごとに違う高さへ動かすという同じ処理を2通りで作り、結果と速さを比べた。

ところが最初は結果が一致しなかった。その理由を追いかけた結果、この2つは「速い遅い」で選ぶものではなく、そもそも別のことをしているのだと分かった。

ノードの接続



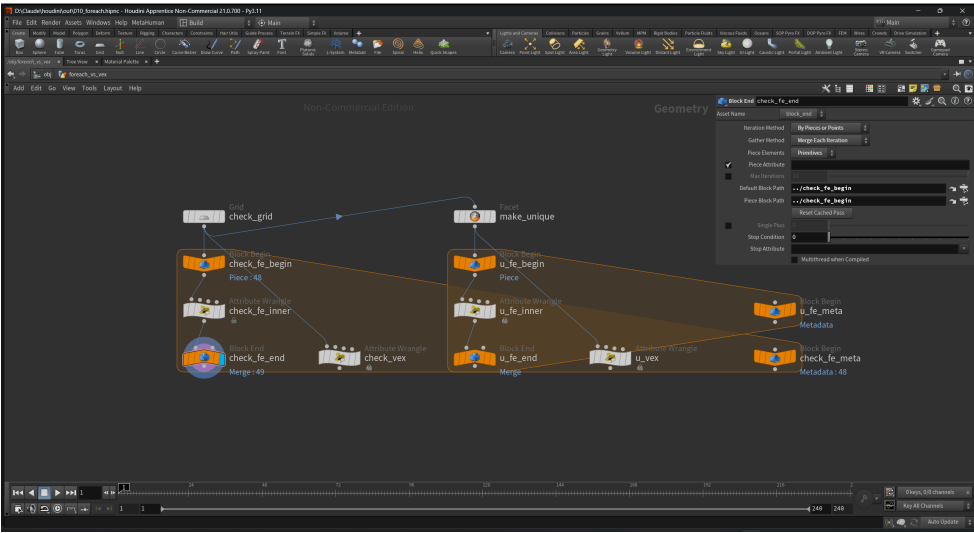
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
check_grid	grid	cols = 8 rows = 8
check_fe_begin	block_begin	blockpath = ../check_fe_end method = 1
check_fe_meta	block_begin	blockpath = ../check_fe_end method = 2
check_fe_inner	attribwrangle	snippet = @P.y += detail("op:../check_fe_meta", "iteration", 0) * 0.001;
check_fe_end	block_end	blockpath = ../check_fe_begin class = 0 itermethod = 1 method = 1 templatepath = ../check_fe_begin
check_vex	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); foreach (int pt; pts) { vector p = point(0, "P", pt); p.y += @primnum * ...
make_unique	facet	unique = 1
u_fe_begin	block_begin	blockpath = ../u_fe_end method = 1
u_fe_meta	block_begin	blockpath = ../u_fe_end method = 2
u_fe_inner	attribwrangle	snippet = @P.y += detail("op:../u_fe_meta", "iteration", 0) * 0.001;
u_fe_end	block_end	blockpath = ../u_fe_begin class = 0 itermethod = 1 method = 1 templatepath = ../u_fe_begin

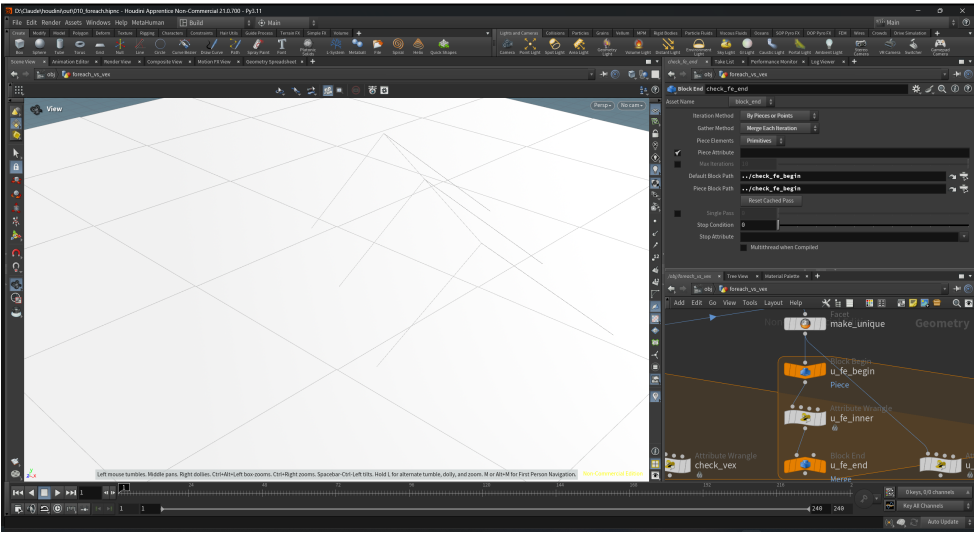
ノード	タイプ	デフォルトから変更したパラメータ
u_vex	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); foreach (int pt; pts) { vector p = point(0, "P", pt); p.y += @primnum * ...

実際の Houdini の画面

for-each は Block Begin と Block End で処理を挟む形で組む。何回目かを知るには、メタデータ用の Block Begin をもう1つ置いて、その中のディテールアトリビュートを読む。



for-each の組み方。左が Block Begin（1個ずつ取り出す）と メタデータ用、中央が処理、右が Block End



Block End のパラメータ。Iteration Method と Piece Elements、そして Template Path

同じ処理にかかる時間（3回の中央値）

面ごとに「何番目か × 0.001」だけ上に動かす処理。for-each は面の数に比例して伸びるが、VEX はほとんど動かない。VEX の 1.2~2.5
ミリ秒はほぼ固定の費用で、実験009で測った「ノード1つあたり約1.8ミリ秒」と一致する。

分割数	プリミティブ数	for-each（ミリ秒）	VEX（ミリ秒）	何倍
8	49	11.79	1.360	8.7倍
16	225	21.08	1.342	15.7倍

分割数	プリミティブ数	for-each (ミリ秒)	VEX (ミリ秒)	何倍
32	961	53.55	1.414	37.9倍
64	3,969	191.87	2.233	85.9倍

結果が一致しなかった理由を追う

同じ処理のはずなのに、49面中48面で高さが違った。しかも差は全部 0.004 で一定。ここから3段階で原因を絞り込んだ。

調べたこと	結果	分かったこと
出力の点数を比べる	元 64点 → for-each 196点 / VEX 64点	for-each は面を1枚ずつ取り出すので、隣と共有していた点が複製される
VEX を3回走らせて結果を比べる	3回とも完全に同じ	複数の面が同じ点に書き込む競合ではあるが、順序は決まっていて再現する
点の共有をほどいてから比べる	完全に一致	食い違いの原因は点の共有。ほどけば2つは同じ結果になる

わかったこと

- ・ for-each は Block Begin (method = Extract Piece or Point) と Block End で処理を挟んで作る。Block End の Template Path が空だと「Missing template geometry」で止まる。何を何回繰り返すかはここから決まる。
- ・ 何回目かを知るには、メタデータ用の Block Begin (method = Fetch Metadata) をもう1つ置き、そのディテールアトリビュート iteration を読む。numiterations で総回数も取れる。
- ・ 取り出した中では @primnum が常に 0 になる。1個だけを切り出した状態で処理しているため。「何番目の面か」を使いたいならメタデータを読むしかない。
- ・ detail() のパスには op: が要る。detail("../メタデータノード", "iteration", 0) と書くとファイルパスとして扱われ、エラーも警告も出さずに 0 を返す。ループが動いていないように見えるので原因が分かりにくい。detail("op:../メタデータノード", ...) と書けば動く。実際にこれで1時間相当を溶かした。
- ・ for-each は面の数に比例して遅くなる。49面で11.79ミリ秒、3,969面で191.87ミリ秒。1面あたりおよそ0.05ミリ秒で、面の数がそのまま効く。
- ・ VEX はほとんど変わらない。49面で1.36ミリ秒、3,969面で2.23ミリ秒。実験009で測った「ノード1つあたり約1.8ミリ秒の固定費」がそのまま出ており、処理そのものの時間はまだ見えていない水準。
- ・ その結果、差は面の数とともに広がる。49面で 8.7倍、3,969面で 85.9倍。面が増えるほど for-each を選ぶ代償が大きくなる。
- ・ ただしこの2つは単純に置き換えられるものではない。for-each は面を1枚ずつ取り出すので、隣と共有していた点が複製される (64点 → 196点)。これは副作用ではなく、面ごとに独立して動かすために必要なこと。点を共有したままでは、隣り合う面が同じ点を別の場所へ動かそうとして取り合いになる。
- ・ その取り合いは競合ではあるが、結果は毎回同じだった (3回とも一致)。書き込む順序が決まっているため、実行のたびに変わることはない。最初は並列処理による非決定性を疑ったが、それは外れた。
- ・ 点の共有をほどいてから (facet の Unique Points で 64点 → 196点にしてから) 比べると、for-each と VEX の結果は完全に一致した。つまり食い違いの原因は処理内容ではなく、点が共有されているかどうか

かだけだった。

- ・実用的な結論。面ごとに独立した変形をしたいなら、まず点の共有をほどこ。ほどいてしまえば VEX で書ける場面が増え、数十倍速い。for-each が必要なのは、1回分の処理が VEX 1つでは書けないほど複雑なとき（中で複数のノードを通したいときなど）。

- ・測り方についても1つ。最初は面の中心を (x, y, z) でソートして比べようとしたが、比べたい量である y をソートの鍵に含めてしまったため、対応がずれて必ず食い違った。比べたい量は鍵にしない。水平位置 (x, z) を鍵にして y を値にしたら正しく比較できた。

次にやること

- ・facet の Unique Points を挟むことで、for-each が必要だった処理をどこまで VEX に置き換えられるかを試す
- ・for-each の中で複数ノードを通す場合（VEX 1つでは書けない場合）の実例を作る
- ・polyextrude と boolean に進む

crease で角を残す — どれだけの重みが必要なのか

2026-09-13 | Houdini 21.0.700 | /obj/crease_test | ノード 15 / 接続 14

何を作ったか

実験002で「subdivide を重ねると立方体が球になる」と測った。3回分割すると一辺が 1.0 から 0.84918 まで縮む。crease（折り目）を付ければ角が保たれるはずだが、どれだけの重みが必要なのかは分からない。

理屈の上では「重み N なら N 回分の分割まで角が保たれる」はず。3回分割するなら重み3で完全に元の立方体のままになる、という予測を立てて確かめた。

結果は予測どおりだった。ただしこの実験では、2つの指標が違う答えを出すという厄介な状況にも当たった。

ノードの接続

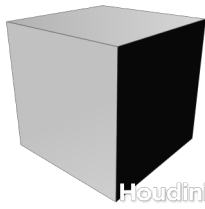


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

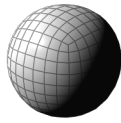
ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
crease_0_0	crease	crease = 0.0 op = 1
sub_0_0	subdivide	iterations = 3
crease_0_5	crease	crease = 0.5 op = 1
sub_0_5	subdivide	iterations = 3
crease_1_0	crease	op = 1
sub_1_0	subdivide	iterations = 3
crease_2_0	crease	crease = 2.0 op = 1
sub_2_0	subdivide	iterations = 3
crease_3_0	crease	crease = 3.0 op = 1
sub_3_0	subdivide	iterations = 3
crease_4_0	crease	crease = 4.0 op = 1
sub_4_0	subdivide	iterations = 3
crease_10_0	crease	crease = 10.0 op = 1
sub_10_0	subdivide	iterations = 3

重みを変えて3回分割する

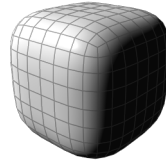
すべて同じカメラ。点数・面数はどの重みでも 386点 / 384面で変わらない。crease は形の計算方法を変えるだけで、ジオメトリを増やさない。



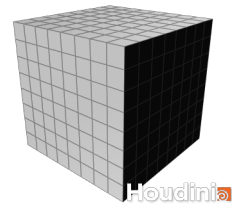
分割前の box 一辺
1.00000 / 折れ角 90.00度



重み 0.0 (crease なし)
一辺 0.84918 / 折れ角
13.93度



重み 1.0 一辺 1.00000 /
折れ角 28.07度

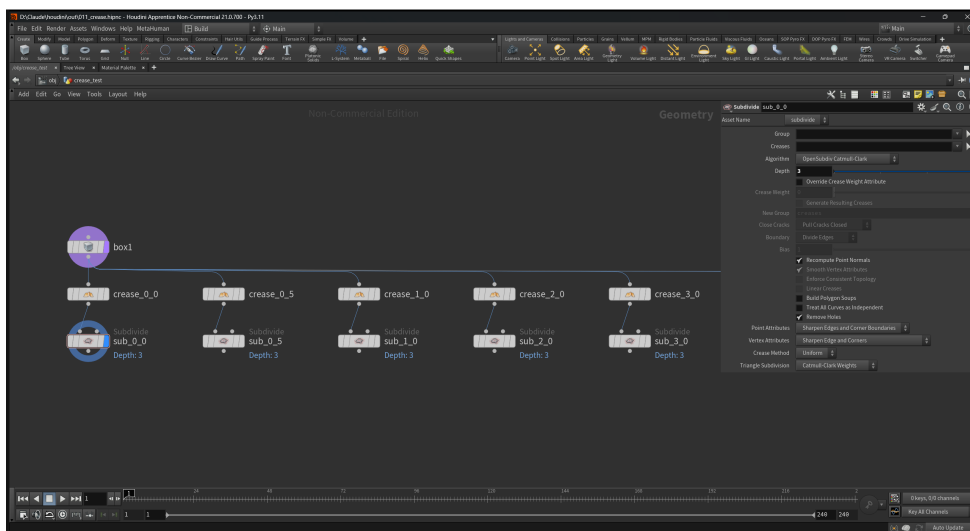


重み 3.0 一辺 1.00000 /
折れ角 90.00度

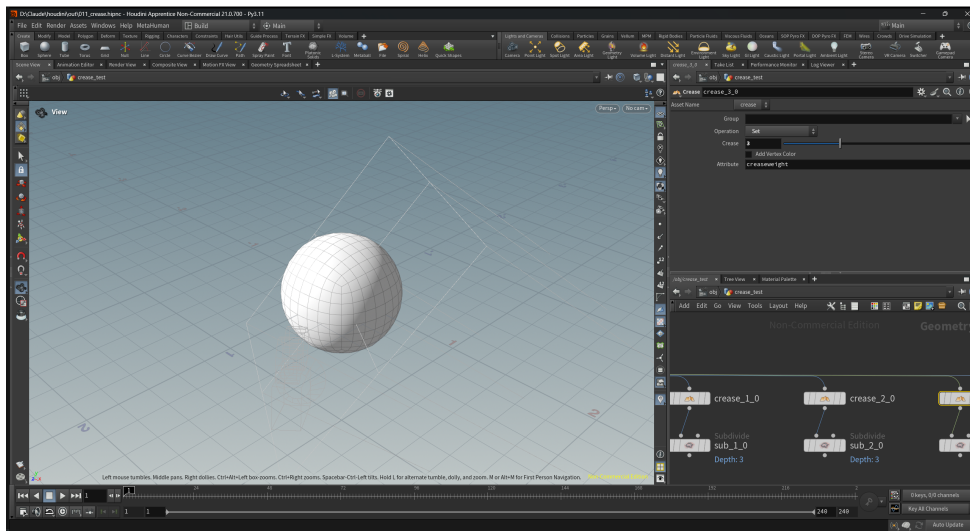
crease の重み	点数	面数	一辺の長さ	最大の折れ角
0.0 (なし)	386	384	0.84918	13.93度
0.5	386	384	0.92459	21.45度
1.0	386	384	1.00000	28.07度
2.0	386	384	1.00000	53.13度
3.0	386	384	1.00000	90.00度
4.0	386	384	1.00000	90.00度
10.0	386	384	1.00000	90.00度
(参考) 分割前の box	8	6	1.00000	90.00度

実際の Houdini の画面

crease SOP で折り目の重みを付け、その下流の subdivide がそれを読む。crease SOP の Crease Attribute の既定は creaseweight で、subdivide はこの名前の属性を自動で見ると。



box から crease と subdivide の組を7通り並べた



crease SOP のパラメータ。Operation を Set にして Crease に重みを入れる

わかったこと

- ・予測は当たった。3回分割する場合、重み 3.0 で折れ角が 90.00度 になり、分割前の box とまったく同じ角の鋭さになる。「重み N なら N 回分の分割まで角が保たれる」という理解でよい。
- ・重みを 3 より上げて何も変わらない (4.0 も 10.0 も 90.00度 のまま)。分割回数を超える重みは無駄。逆に言えば、分割回数を後から増やすつもりなら重みも合わせて上げる必要がある。
- ・点数・面数はどの重みでも 386点 / 384面で変わらなかった。crease は形の計算方法を変えるだけで、ジオメトリを増やさない。角を残すためにポリゴンを足す必要はない。
- ・2つの指標が違う答えを出した。一辺の長さは重み 1.0 で 1.00000 に達し、そこから変わらない。この指標だけを見ていたら「重み1で十分」と判断してしまう。しかし折れ角は 28.07度 で、まだ元の 90度 には程遠い。画を見ると角がはっきり丸い。
- ・つまり一辺の長さは「どこまで縮んだか」しか測っておらず、「角が鋭いかどうか」は測れていない。重み1で外形は元に戻るが、角は丸いまま。何を知りたいかによって、見るべき指標が変わる。
- ・折れ角の指標は、辺を共有する2つの面の法線のなす角の最大値として作った。角が残っていれば大きく、丸まれば小さくなる。分割前の box で 90.00度 と出るので、基準としても正しく働いている。
- ・重み 0.5 のとき一辺は 0.92459 だった。これは crease なし (0.84918) と重み1.0 (1.00000) のちょうど中間で、小数点以下5桁まで一致する。小数の重みは2つの結果を線形に混ぜている。
- ・ただし折れ角のほうは線形ではない。0.0 / 0.5 / 1.0 で 13.93 / 21.45 / 28.07度 となり、中間値の 21.00度 とはわずかにずれる。位置は線形に混ざるが、角度はそうならない。
- ・crease SOP の Operation の既定は「Add To Existing Value」で、既にある値に足し込む。重みを決め打ちしたいなら「Set」にする。既定のまま何度も通すと重みが積み上がる。
- ・crease SOP が書き込む属性名の既定は creaseweight で、subdivide はこの名前を自動で見る。名前を変えたい場合は subdivide 側の Creases にも同じ名前を指定する必要がある。

次にやること

- ・一部の辺だけに crease を付けて、角を残す場所と丸める場所を作り分ける
- ・polyextrude で作った形に crease を付け、建物のような硬い形を作る

- ・折れ角の指標を、面ごとではなく辺ごとに出して分布を見る（どの辺が丸まっているかを特定する）

polyextrude と boolean — 硬い形を作る2つの道具

2026-09-13 | Houdini 21.0.700 | /obj/extrude_boolean | ノード 15 / 接続 20

何を作ったか

ここまではノイズで有機的な形を作ってきた。建物や機械のような硬い形は別の道具で作る。面を押し出す polyextrude と、立体どうしを足し引きする boolean。

押し出しは距離が指定どおりになるのか、点数はどう増えるのか。boolean は演算ごとに何が変わるのか。そして boolean が正しく計算できているかを、体積の関係式で検算した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
grid	grid	cols = 5 rows = 5
ext_0_0	polyextrude::2.0	thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5
ext_0_5	polyextrude::2.0	dist = 0.5 thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5
ext_1_0	polyextrude::2.0	dist = 1.0 thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5

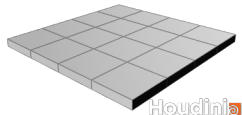
ノード	タイプ	デフォルトから変更したパラメータ
ext_2_0	polyextrude::2.0	dist = 2.0 thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5
bool_box	box	scale = 2.0
bool_sphere	sphere	cols = 30 radx = 1.35 rady = 1.35 radz = 1.35 rows = 30 type = 1
bool_union	boolean::2.0	(変更なし)
bool_intersect	boolean::2.0	booleanop = 1
bool_subtract	boolean::2.0	booleanop = 2
bool_shatter	boolean::2.0	booleanop = 3
bool_custom	boolean::2.0	booleanop = 4
bool_seam	boolean::2.0	booleanop = 6
bool_detect	boolean::2.0	booleanop = 8
bool_resolve	boolean::2.0	booleanop = 9

polyextrude の距離

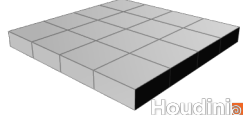
10×10 の grid (5分割 = 25点 / 16面) を押し出した。距離を 0 にすると何も起きず、0 より大きければ点数・面数は距離によらず一定になる。



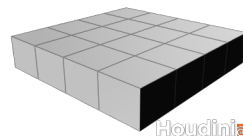
距離 0.0 25点 / 16面



距離 0.5 41点 / 32面



距離 1.0 41点 / 32面



距離 2.0 41点 / 32面

距離	点数	面数	できた高さ	指定との差
0.0	25	16	0.00000	0.00000
0.5	41	32	0.50000	0.00000
1.0	41	32	1.00000	0.00000
2.0	41	32	2.00000	0.00000

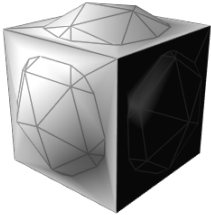
inset (内側への縮み)

距離 1.0 で固定して inset だけ変えた。上面がどれだけ小さくなるかを測っている。元の一辺は 10.0。

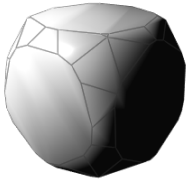
inset	点数	面数	上面の一辺	元との差
0.0	41	32	10.00000	0.0
0.3	41	32	9.40000	0.6 = 0.3 × 2
0.8	41	32	8.40000	1.6 = 0.8 × 2

boolean の演算ごとの結果

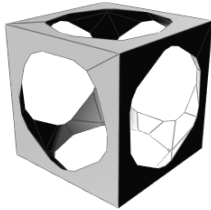
2×2×2 の box（体積 8.00000）と、半径 1.35 のポリゴン球（体積 9.00180）を組み合わせた。



Union（和集合） 体積 9.91613



Intersect（積集合） 体積 7.08567



Subtract（差集合） 体積 0.91433

演算	点数	面数	体積
Union（和集合）	130	120	9.91613
Intersect（積集合）	96	60	7.08567
Subtract（差集合）	104	60	0.91433
Shatter（砕く）	200	120	8.00000
Custom	200	120	8.00000
Seam（境界線）	88	6	0.00000
Detect（検出）	8	6	8.00000
Resolve（重なるの解消）	152	496	2.91612

boolean が正しく計算できているかの検算

集合の大きさには必ず成り立つ関係がある。これを体積で確かめれば、boolean が正しく動いているかを数値で検証できる。

確かめた式	左辺	右辺	差
和集合 + 積集合 = box + 球	17.00180	17.00180	0.0000049
差集合 = box - 積集合	0.91433	0.91433	0.0000000

わかったこと

・ polyextrude の距離は指定どおりに出る。0.5 / 1.0 / 2.0 を指定して、できた高さは 0.50000 / 1.00000 / 2.00000 で差はゼロ。これは当たり前に見えて重要で、ノイズ系のパラメータ（height など）が種類によって実際のずれが9倍も違ったのとは対照的。押し出しは寸法をそのまま指定できる。

- ・点数・面数は距離によらず一定（41点 / 32面）。距離 0 のときだけ何も起きない（25点 / 16面のまま）。形を変えても重さは変わらない。
- ・元の 25点 / 16面 が 41点 / 32面 になる内訳も説明できる。5×5 の grid は境界に 16点あり、押し出すとその 16点が複製されて $25 + 16 = 41$ 点。面は上面 16 + 側面 16 = 32面。
- ・inset は各辺から内側に引っ込む距離。0.3 にすると一辺が両側から 0.3 ずつで 0.6 縮み、10.0 → 9.40000 になった。0.8 なら 1.6 縮んで 8.40000。指定した値がそのまま距離として効く。
- ・パラメータ名は dist と inset。分割数は divisions ではなく divs。ここでも一般的な名前ではなく実際の名前を確認する必要があった。
- ・boolean には8つの演算がある。よく使う Union / Intersect / Subtract のほかに、Shatter（Aを砕いて全部の破片を残す）、Seam（境界線だけを出す）、Detect（検出だけして形は変えない）、Resolve（自分自身の重なりを解消する）がある。
- ・Shatter の体積は 8.000000 で box と完全に一致した。砕いても全部の破片が残るので体積は変わらない。破壊エフェクトの下準備がこれ。
- ・Seam は面数 6 のまま体積 0 になる。境界線だけを取り出す演算なので立体にならない。
- ・体積の関係式で検算できた。集合には「和集合 + 積集合 = 元の2つの合計」という関係が必ず成り立つ。実測すると 17.00180 対 17.00180 で差は 0.0000049。また「差集合 = box - 積集合」も 0.91433 対 0.91433 で差は完全にゼロ。boolean が正しく計算できていることを数値で確認できた。
- ・この検算は、形が正しいかどうかを目で見ずに確かめられる点で価値がある。複雑な boolean の結果が正しいか不安なときは、体積の関係を見れば判断できる。
- ・sphere SOP で rows / cols を 30 に設定したが、面数は 80 のままだった。ポリゴン球は rows / cols ではなく別の分割方法（多面体の細分）を使っている。分割数を上げたいなら別のパラメータを探す必要がある。
- ・その粗い球（80面）の体積は 9.00180 だった。半径 1.35 の真球なら 10.30575 なので、約 13% 小さい。ポリゴンで近似した球は必ず内側に寄るので、体積を当てにするなら分割数を上げる必要がある。

次にやること

- ・polyextrude を繰り返して階層のある建物を作る。for-each との組み合わせ
- ・sphere の分割数を上げる方法を調べ、体積が真球にどう近づくかを測る
- ・エフェクトに入るための下準備（複数フレームの書き出し、シミュレーションのキャッシュ）

時間軸への対応 — フレームを進めて記録する

2026-09-13 | Houdini 21.0.700 | /obj/time_axis | ノード 4 / 接続 2

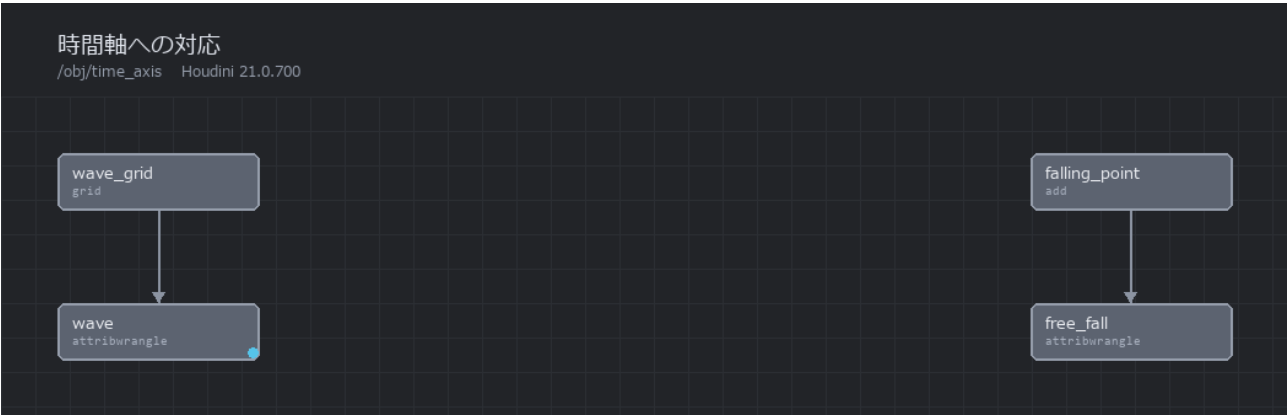
何を作ったか

ここまでの12実験はすべて1フレームだけを見ていた。エフェクトは時間変化そのものが結果なので、フレームを進めて連番で書き出し、1枚のコンタクトシートと動くGIFにまとめる仕組みが要る。

仕組みが正しいかは、答えが分かっている動き（自由落下の式）で確かめた。

あわせて、この先のシミュレーションで使うノードがどこにあるのかを数えた。その結果、当初の予定を変えるべきだと分かった。

ノードの接続

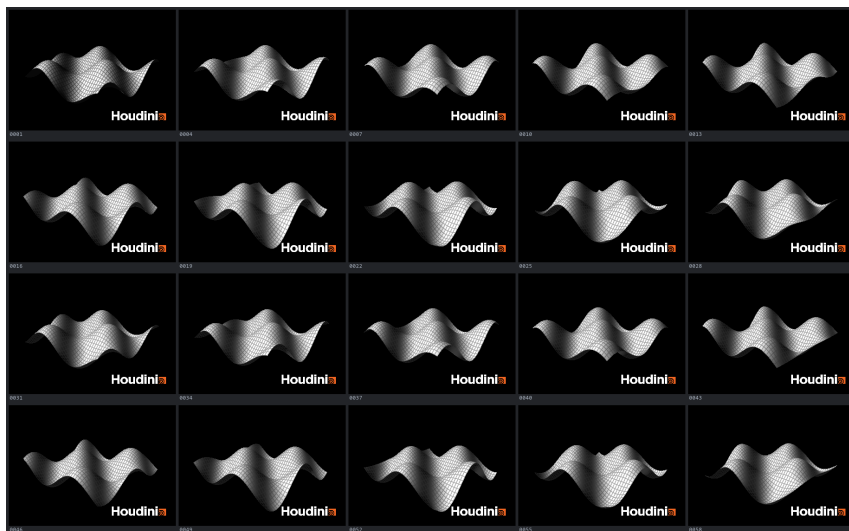


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
wave_grid	grid	cols = 40 rows = 40
wave	attribwrangle	snippet = @P.y = sin(@P.x * 0.8 + @Frame * 0.2) * cos(@P.z * 0.8) * 1.5;
falling_point	add	points = 1
free_fall	attribwrangle	snippet = float t = (@Frame - 1) / 24.0; @P.y = 20.0 - 0.5 * 9.8 * t * t;

連番の書き出し（20コマ）

フレーム1から58まで3コマおき。カメラは全フレームを通して固定してある。動いているのか大きさが変わっているのかを画から判断できるようにするため。



20コマを1枚に並べたコンタクトシート。左下の数字がフレーム番号

検証: 自由落下の式と一致するか

高さ20から重力9.8で落ちる点を作り、理論値 $h = \frac{1}{2}gt^2$ と比べた。24コマで1秒として計算している。

フレーム	経過秒	測った高さ	理論値	差
1	0.0000	20.00000	20.00000	0.0000000
10	0.3750	19.31094	19.31094	0.0000004
20	0.7917	16.92899	16.92899	0.0000002
30	1.2083	12.84566	12.84566	-0.0000005
40	1.6250	7.06094	7.06094	-0.0000006

シミュレーション関連ノードがどこにあるか

SOP は形を作る階層、DOP はシミュレーションの階層。どちらに何種類あるかを数えると、どこから手を付けるべきかが見えてくる。

対象	キーワード	SOP にある数	DOP にある数
パーティクル	pop	0 種	67 種
剛体（破壊）	rbd	47 種	21 種
布	vellum	22 種	7 種
煙・炎	pyro	14 種	3 種
液体	flip	8 種	4 種

わかったこと

- ・連番の書き出しは正しく動いている。自由落下の式との差は最大でも 0.0000006 で、これは小数の計算精度そのもの。フレームを進めて記録する仕組みに間違いはない。
- ・20枚の書き出しに 1.18 秒（1枚あたり 59 ミリ秒）。60フレーム全部でも3.5秒程度で、実用的な速さ。レンダリングの重さは問題にならない。
- ・カメラは全フレームを覆う範囲で一度だけ決める。フレームごとに枠を合わせ直すと、対象が動いているのか大きさが変わっているのかを画から判断できなくなる。そのために、全フレームを走査して範囲を

求める関数を用意した。

- ・出力は2つの形にした。PDFには動きを載せられないのでコンタクトシート（全コマを1枚に並べてフレーム番号を振る）、HTMLにはGIF。同じ連番から両方を作る。

- ・パーティクルは SOP に1つもない（0種）。すべて DOP

側にある（67種）。つまりパーティクルを扱うには DOP

ネットワークを組む必要があり、他の実験のようにSOPをつなぐだけでは始められない。

- ・一方で剛体（RBD）は SOP に 47種もある。布（Vellum）も

22種。SOPレベルのラッパーが充実している分野の方が、入口としては易しい。

当初はパーティクルから始める予定だったが、RBD か Vellum から入る方が理にかなっている。

- ・煙・炎（Pyro）は SOP 14種 / DOP 3種、液体（FLIP）は SOP 8種 / DOP 4種。どちらも SOP 側にラッパーはあるが数は少ない。

- ・ solver SOP

を使って自分で重力を書こうとしたが、これはロックされたアセットで、中にノードを作るには `allowEditingOfContents()` が必要。しかも中身は `dopnet` と4つの `null` で、想像していた「前フレームの結果を受け取って加工する」という単純な構造ではなかった。プログラムから組むには追加の調査が必要

。

- ・そのため今回は、自由落下を数式で書いて仕組みの検証にとどめた。これはシミュレーションではない（前のフレームの結果を使っていない）。本物のシミュレーションは次の実験から。

次にやること

- ・ RBD（剛体）から始める。SOPレベルのノードが47種あり、boolean の Shatter で作った破片をそのまま落とせる

- ・ Vellum（布）も SOPレベルのノードが揃っているので入りやすい

- ・シミュレーションの結果をファイルに保存して使い回す（キャッシュ）。毎回計算し直すと実験が遅くなる

RBD 破壊 — 最初の本物のシミュレーション

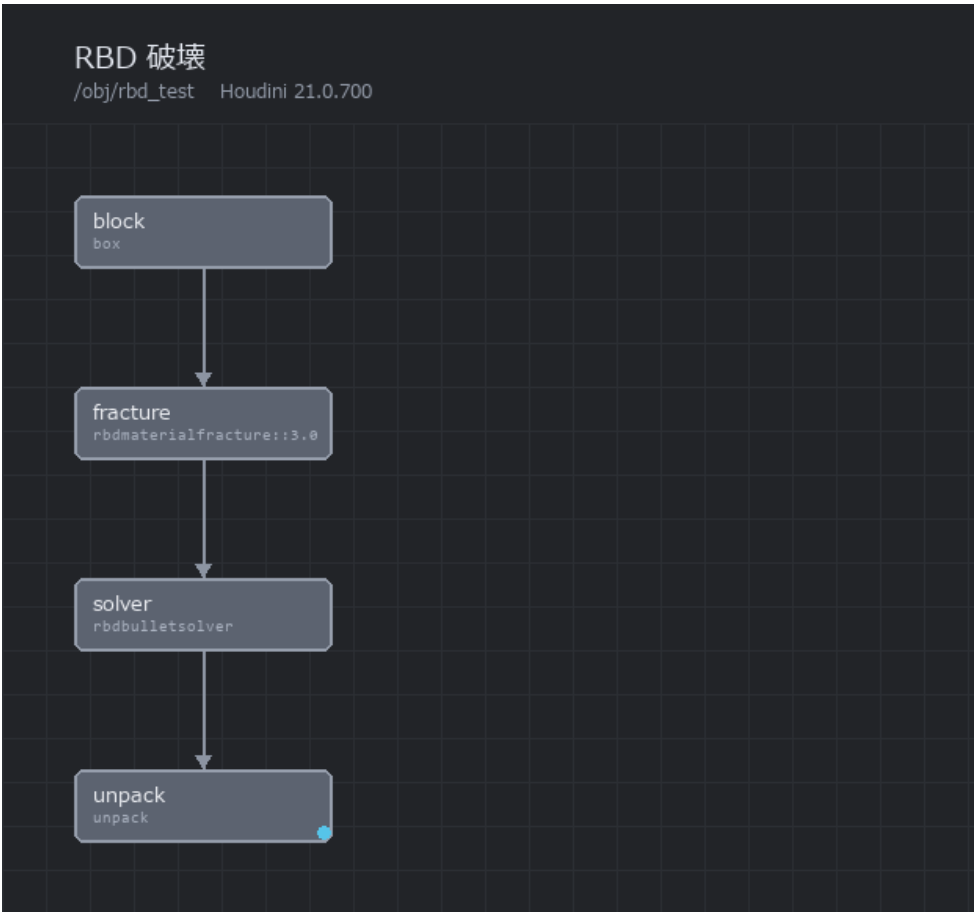
2026-09-13 | Houdini 21.0.700 | /obj/rbd_test | ノード 4 / 接続 3

何を作ったか

実験013で、剛体（RBD）は SOP レベルのノードが47種あり、パーティクルより入口が易しいと分かった。ここまでの実験はすべて「1フレームの計算」だったが、今回は前のフレームの結果を次に渡していく本物のシミュレーション。

シミュレーションは結果が複雑で、目で見ても正しいかどうか判断しづらい。そこで検証の軸を1つ決めた。剛体は変形しないので、砕けても落ちてもぶつかっても、全部の破片の体積の合計は変わらないはず。これが崩れたら計算のどこかがおかしい。

ノードの接続



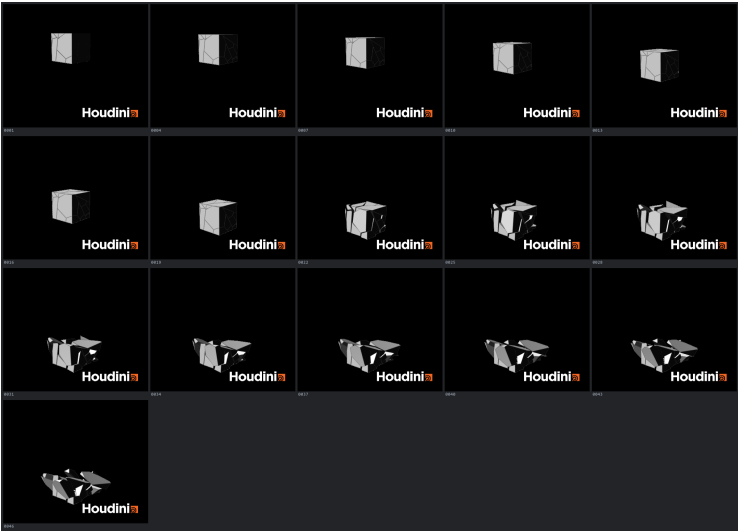
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
block	box	scale = 2.0 ty = 4.0
fracture	rbdmaterialfracture::3.0	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
solver	rbdbulletsolver	folder10 = 1 folder13 = 1 folder21 = 1 folder24 = 1 folder26 = 1 folder26_1 = 1 folder26_2 = 1 folder27 = 1 folder3 = 1 folder34 = 1 folder35 = 1 folder38 = 1 …ほか 11 件
unpack	unpack	(変更なし)

高さ4から落として砕く（16コマ）

2×2×2 の箱を rbdmaterialfracture（コンクリート）で25個の破片に砕き、rbdbulletsolverで落とした。地面はソルバの機能で用意している。



16コマ。落下中は塊のまま、着地して初めて砕ける

フレームごとの状態

「横の広がり」はバウンディングボックスの水平方向の大きさ。元の箱は 2.0 なので、これが増えていけば破片が散らばったということ。

フレーム	体積の合計	最下点	横の広がり	高さ	計算にかかった秒
1	8.00000	3.0000	2.0000	2.0000	0.38
4	8.00000	2.9208	2.0000	2.0000	0.02
10	8.00000	2.3028	2.0000	2.0000	0.03
20	8.00000	−0.0492	2.0589	2.0331	0.05
30	8.00000	−0.0225	3.7483	2.0181	0.03
46	8.00000	−0.0219	5.6408	1.9498	0.05

検証: 体積は保たれているか

調べたこと	結果
元の箱 (2×2×2) の体積	8.00000
砕いた直後の合計	8.00000 (差 0.00000)
全フレームでの振れ幅	0.0000000 (0.0000%)

わかったこと

- ・体積は完全に保たれた。全6フレームで 8.00000 ちょうど、振れ幅は 0.0000000。元の箱の体積 8.0 とともに完全に一致する。砕いても落としてもぶつかっても体積が変わらないので、剛体シミュレーションとして正しく動いている。
- ・これは目で見ても分からない種類の検証。破片が散らばった画を見ても「正しく計算できているか」は判断できないが、体積が保たれていれば少なくとも物質は増減していないと言える。実験012で boolean に使った検算と同じ考え方。
- ・落下中は砕けない。フレーム1から10まで、横の広がり 2.0000 のまま完全に変化しない。砕いてはあるが、破片はくっついたまま1つの塊として落ちる。
- ・着地の瞬間に広がり始める。フレーム20で最下点が -0.0492 になり（地面を少し突き抜ける）、同時に広がりが 2.0589 に増える。その後 30で 3.7483、46で 5.6408。元の 2.8倍まで散らばった。
- ・同時に全体の高さは 2.0000 から 1.9498 に下がる。積み上がっていたものが平たく崩れたということ。
- ・最下点は着地時に -0.0492 まで沈み、その後 -0.0219 で落ち着く。破片は地面を少し突き抜けたまま止まる。ソルバが衝突判定に持たせている余裕（マージン）によるもので、厳密に地面の上に乗るわけではない。
- ・rbdmaterialfracture のコンクリートモードで、2×2×2 の箱が 25個の破片になった（282点 / 191面）。破片の区別は name というプリミティブ属性で管理されている。
- ・計算は速かった。最初のフレームが 0.38秒、以降は 0.02～0.05秒。全部で1秒程度。シミュレーションといっても、この規模なら実験を回すのに支障はない。
- ・rdbbulletsolver の既定は substeps = 10、cacheenabled = 1。1フレームを10回に分けて計算しており、結果は自動でキャッシュされる。2回目以降のフレーム送りが速いのはこのため。
- ・地面はソルバの useground で用意できる。別途ジオメトリを作る必要はない。

次にやること

- ・substeps を減らすと結果がどう変わるか。突き抜けや不安定さが出るはず
- ・破片の数を増やしたときの計算時間の伸び方を測る
- ・Vellum（布）でも同じように、保存されるべき量を見つけて検証する

Vellum クロス — 布で保存されるべき量は何か

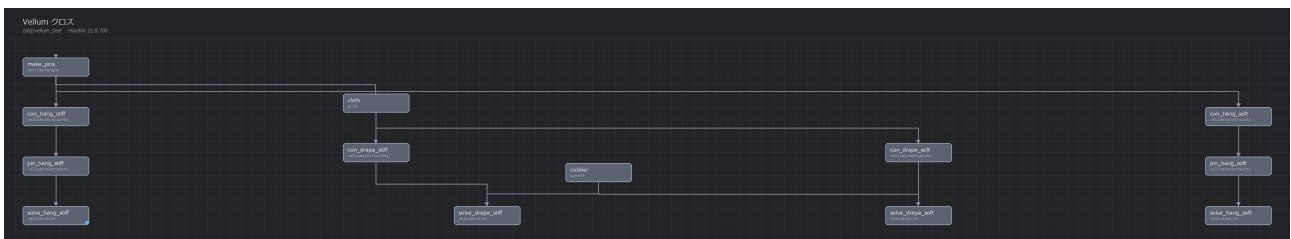
2026-09-13 | Houdini 21.0.700 | /obj/vellum_test | ノード 13 / 接続 19

何を作ったか

剛体では体積が保たれた（実験014）。布は面なので体積では測れない。代わりに「辺の長さの合計」を見る。布は伸びにくい素材なので、たわんでも吊るしても、辺の長さの合計はほぼ変わらないはず。

さらに、伸びにくさ (stretchstiffness) を下げれば保存が崩れるはず。そう考えて試したところ、まったく変化しなかった。原因を追ったら、このパラメータの本当の仕組みが分かった。

ノードの接続



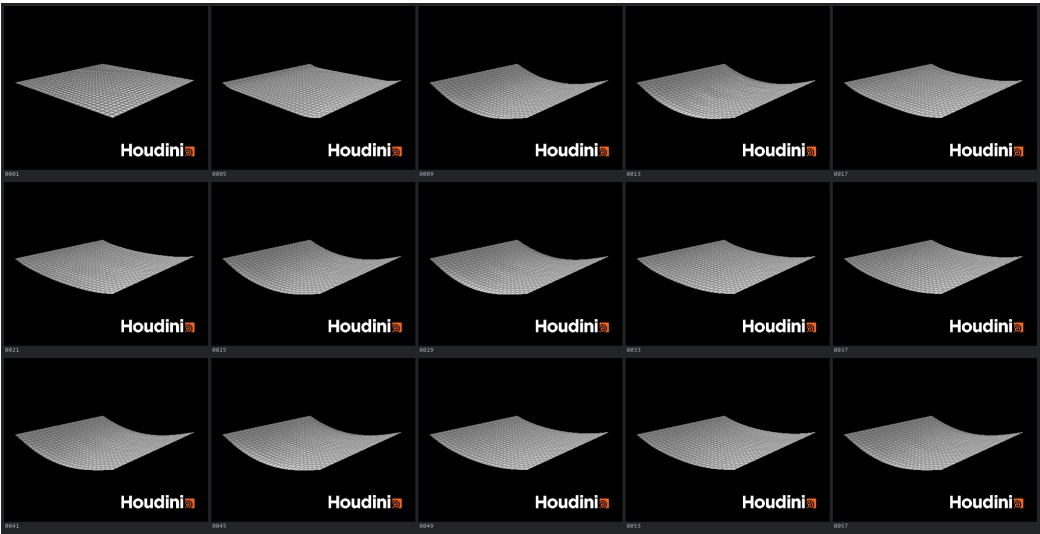
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
cloth	grid	cols = 30 rows = 30 sizeX = 4.0 sizeY = 4.0 ty = 3.0
collider	sphere	radX = 1.2 radY = 1.2 radZ = 1.2 ty = 1.0 type = 1
make_pins	attribwrangle	snippet = if (abs(@P.x) > 1.9) { @group_pins = 1; }
con_drape_stiff	vellumconstraints	constrainttype = 3
solve_drape_stiff	vellumsolver	useground = 1 windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
con_drape_soft	vellumconstraints	constrainttype = 3 stretchstiffness = 0.02
solve_drape_soft	vellumsolver	useground = 1 windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
con_hang_stiff	vellumconstraints	constrainttype = 3
pin_hang_stiff	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve_hang_stiff	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618

ノード	タイプ	デフォルトから変更したパラメータ
con_hang_soft	vellumconstraints	constrainttype = 3 stretchstiffness = 0.02
pin_hang_soft	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve_hang_soft	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618

両端をピンで留めて吊るす（15コマ）

4×4 の布（900点 / 841面）の両端をピンで固定し、自重でたわませた。ピンで留めると布に張力がかかるので、伸びにくさの違いが出やすくなる。



15コマ。両端が固定されたまま中央が垂れ下がる

条件ごとの伸び率

「伸び率」は辺の長さの合計を、たわむ前の状態（464.00000）で割った値。1.0 なら一切伸びていない。

条件	最大の伸び率	伸びた割合
たわむ・かたい（stiffness 1.0）	1.00272	0.272%
たわむ・やわらかい（stiffness 0.02）	1.00257	0.257%
吊るす・かたい（stiffness 1.0）	1.00842	0.842%
吊るす・やわらかい（stiffness 0.02）	1.00842	0.842%

かたさを大きく振る（吊るした布、最終フレーム）

stiffness を 50倍変えても結果が同じだったので、指数のパラメータを含めて広く振った。値と指数を入れ替えたペアの結果が完全に一致したので、実効的なかたさは「値 × 10の指数乗」だと確定した。

stretchstiffness	stretchstiffnessexp	実効的なかたさ	最後の伸び率	最下点
1.0	6	10 ⁶	1.00843	2.6042

stretchstiffness	stretchstiffnessexp	実効的なかたさ	最後の伸び率	最下点
1000000.0	0	10^6	1.00843	2.6042
1.0	0	10^0	2.74029	-3.9926
1.0	-6	10^{-6}	3.53114	-5.7265
0.000001	0	10^{-6}	3.53114	-5.7265

わかったこと

- ・点数は全フレームで 900 のまま変わらない。布は形が変わるだけで、点が増減することはない。剛体の体積保存にあたるのが、布ではこの「構造が変わらないこと」。
- ・たわませただけでは、辺の長さの合計はほとんど変わらなかった（最大 0.272% の伸び）。布が自分の重さで垂れるだけでは、張力がほとんどかからないため。
- ・両端をピンで留めて吊ると 0.842% まで増えた。たわむ場合の3.1倍。布を引っ張る力がかかっている分だけ伸びている。指標が張力を捉えられていることが確認できた。
- ・ところが、かたさを 1.0 から 0.02 に下げても結果がまったく変わらなかった。50分の1にしたのに伸び率は 1.00842 で小数点以下5桁まで同じ。
- ・原因は stretchstiffnessexp というパラメータで、既定値が 10 だった。実効的なかたさは「値 × 10の指数乗」なので、既定では 1.0×10^{10} という極端に硬い状態。ここから 0.02 にしても 2×10^8 で、やはり伸びない。
- ・これを確かめるために値と指数を広く振った。(値 1.0, 指数 6) と (値 1000000, 指数 0) の結果が完全に一致し、(値 1.0, 指数 -6) と (値 0.000001, 指数 0) も完全に一致した。掛け算であることが実測で確定した。
- ・指数を 0 まで下げると伸び率 2.74029 (174%の伸び) 、-6 まで下げると 3.53114 (253%の伸び) になり、布というよりゴムのような挙動になる。最下点も 2.60 から -5.73 まで下がる。
- ・実用上の意味は大きい。画面に見えている数字 (1.0) だけを触っても、実際にはほとんど変わらない。効くのは指数のほう。布が硬すぎる・柔らかすぎると感じたなら、値ではなく指数を疑う。
- ・ピンの指定でも1つつまずいた。group type の既定はプリミティブなので、点のグループを渡すなら先に points へ切り替える必要がある。さらに group を空のままにすると「全部が対象」と解釈され、布全体がピン留めされて一切動かなくなる。最下点が全フレーム 3.0000 のまま動かないという分かりやすい症状が出た。
- ・計算は速く、1フレームあたり 0.3~0.8秒。900点の布ならシミュレーションでも実験のテンポは保てる。

次にやること

- ・ bendstiffness にも同じ指数があるか確かめる。あるなら曲がりにくさも同じ罫がある
- ・布と球の衝突で、厚み (thickness) がどう効くかを測る
- ・Pyro (煙) へ進む。ボリウムなので、保存されるべき量がまた変わる

Pyro 煙 — dissipation の正体を式で突き止める

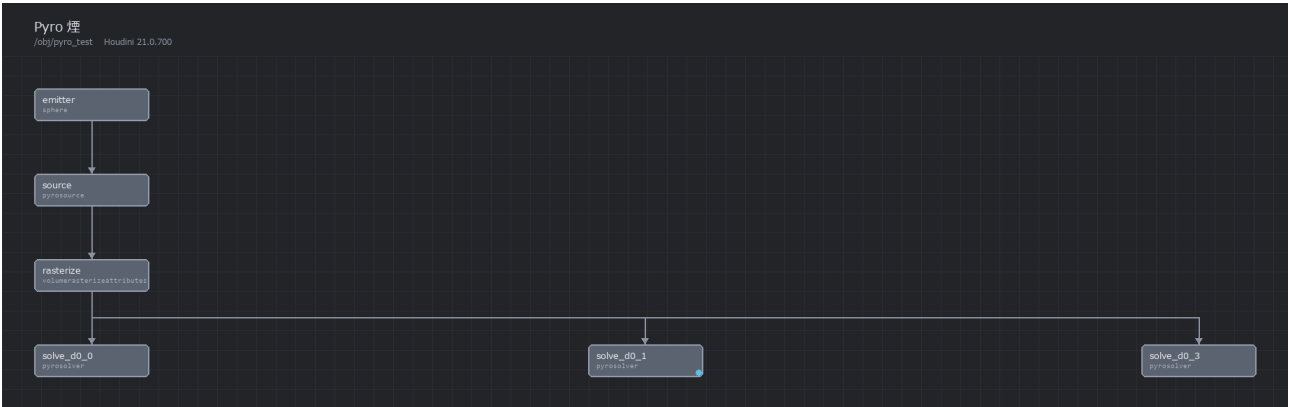
2026-09-13 | Houdini 21.0.700 | /obj/pyro_test | ノード 6 / 接続 5

何を作ったか

剛体は体積、布は辺の長さが保たれた。煙はボリウム（空間を格子で区切って各マスに値を持たせたもの）なので、測るのは密度の合計になる。

当初は「煙は消えていくので密度は減るはず」と考えたが、実際には増えた。発生源が毎フレーム供給し続けるためで、減衰と供給が釣り合う。そこで「供給と減衰の釣り合い」を式に立てて実測と突き合わせたところ、小数点以下4桁まで一致した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.5 rady = 0.5 radz = 0.5 ty = 0.6 type = 1
source	pyrosource	attribute1 = 0 attributes = 1 mode = 2
rasterize	volumerasterizeattributes	attributes = density folder1 = 1 folder3 = 1 voxelsize = 0.08
solve_d0_0	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 102 件

ノード	タイプ	デフォルトから変更したパラメータ
solve_d0_1	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 100 件
solve_d0_3	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.3 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 101 件

密度の合計の変化（最初のフレームを1とした比）

半径0.5の球から煙を出し続けた。dissipation は煙が消える速さのパラメータで、既定は 0.1。

フレーム	散逸なし (0.0)	既定 (0.1)	強め (0.3)
1	1.00000	1.00000	1.00000
5	5.00000	4.09489	2.77298
10	10.00000	6.51284	3.23902
20	20.00000	8.78371	3.33052
30	30.00000	9.57551	3.33310
46	46.00000	9.92084	3.33318

立てた式と実測の突き合わせ

供給が毎フレーム一定で、毎フレーム $(1 - d)$ 倍に減るとすれば、 t フレーム後の合計は $(1 - (1-d)^t) / d$ 倍になるはず。この式と実測を比べた。

条件	フレーム	実測の比	式の予測	差
散逸なし	10	10.00000	10.00000	0.000000
散逸なし	46	46.00000	46.00000	0.000000
既定 0.1	5	4.09489	4.09510	-0.000210
既定 0.1	20	8.78371	8.78423	-0.000523
既定 0.1	46	9.92084	9.92145	-0.000608

条件	フレーム	実測の比	式の予測	差
強め 0.3	5	2.77298	2.77310	-0.000120
強め 0.3	46	3.33318	3.33333	-0.000153

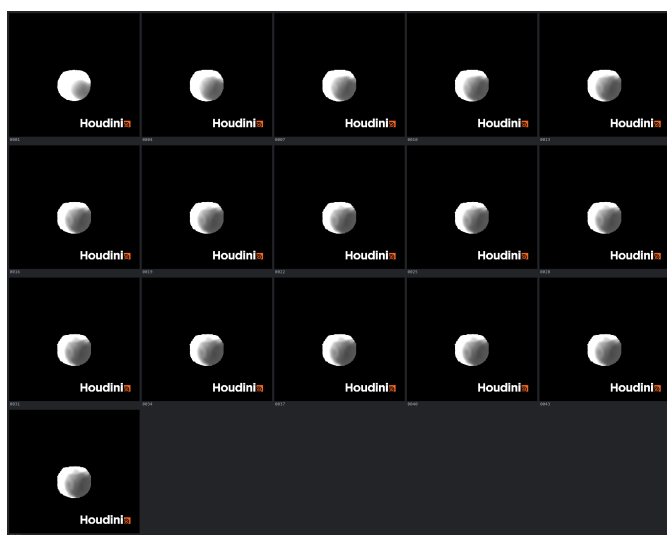
行き着く先（釣り合いの値）

供給と減衰が釣り合うと、密度はある値で止まる。式の上では $1 / \text{dissipation}$ になるはず。

条件	最後の実測	$1 / \text{dissipation}$	差
既定 0.1	9.92084	10.00000	0.079
強め 0.3	3.33318	3.33333	0.00015

見た目（16コマ）

数値ははっきりしているが、画はあまり情報を持っていない。理由は下の「わかったこと」に書いた。



16コマ。ハードウェアレンダラーはポリウムを塊としてしか描けない

わかったこと

- ・当初の予想は外れた。「煙は消えていくので密度は減る」と考えていたが、実際には増えた。発生源が毎フレーム煙を供給し続けるため、減る量より足される量が多い間は増え続ける。
- ・散逸なし（ $\text{dissipation} = 0$ ）のとき、密度の合計はフレーム番号とぴったり同じ比になった。5フレーム目で5.000000倍、46フレーム目で46.000000倍。差は完全にゼロ。毎フレーム同じ量が足され、一切減らないので当然だが、これがきれいにすることでシミュレーションが余計なことをしていないと確認できる。
- ・ dissipation は「毎フレーム $(1 - d)$ 倍に減らす」という意味だと確定した。この仮定から立てた式 $(1 - (1-d)^t) / d$ と実測を比べると、全フレームで差が 0.0006 以下（最大でも 0.006%）。
- ・行き着く先も式のとおり。供給と減衰が釣り合う値は $1 / \text{dissipation}$ で、0.3 のとき理論値 3.33333 に対し実測 3.33318。0.1 のときは 10 に対し 9.92084 で、まだ完全には収束していないが確実に近づいている。
- ・この形の検証は応用が効く。パラメータの意味が分からないときは、仮定を式にして実測と突き合わせればよい。説明を読むより確実に、桁が合っているかまで分かる。実験015で「かたさ = 値 × 10の指数乗」を突き止めたのと同じやり方。

- ・ボリウムの構成でつまづいた点。pyrosource は既定で P と pscale しか作らず、density 属性を明示的に追加しないと次のラスライズが空を返す。そして紛らわしいことに、pyrosource の attributes は「作る属性の個数」（数値）、volumerasterizeattributes の attributes は「属性名を並べた文字列」で、同じ名前なのに意味が違う。

- ・ハードウェアレンダラー（OpenGL ROP）はボリウムの描画に向いていない。煙が塊のようにしか描かれず、濃淡も立ち上りも分からない。これまでの実験ではこのレンダラーで十分だったが、ボリウムを扱うならKarmaのような本式のレンダラーが要る。

- ・計算は速く、1フレームあたり 0.04~0.2秒。ボリウムでもこの規模なら実験のテンポは保てる。

次にやること

- ・Karma でボリウムをレンダリングし、煙らしい見た目を出す。レンダリング時間も測る
- ・buoyancylift（浮力）を変えて、立ち上る速さがどう変わるかを測る
- ・FLIP（液体）へ。液体なら体積が保存されるはずなので、また別の検証軸になる

FLIP 液体 — 組み方が分からず保留

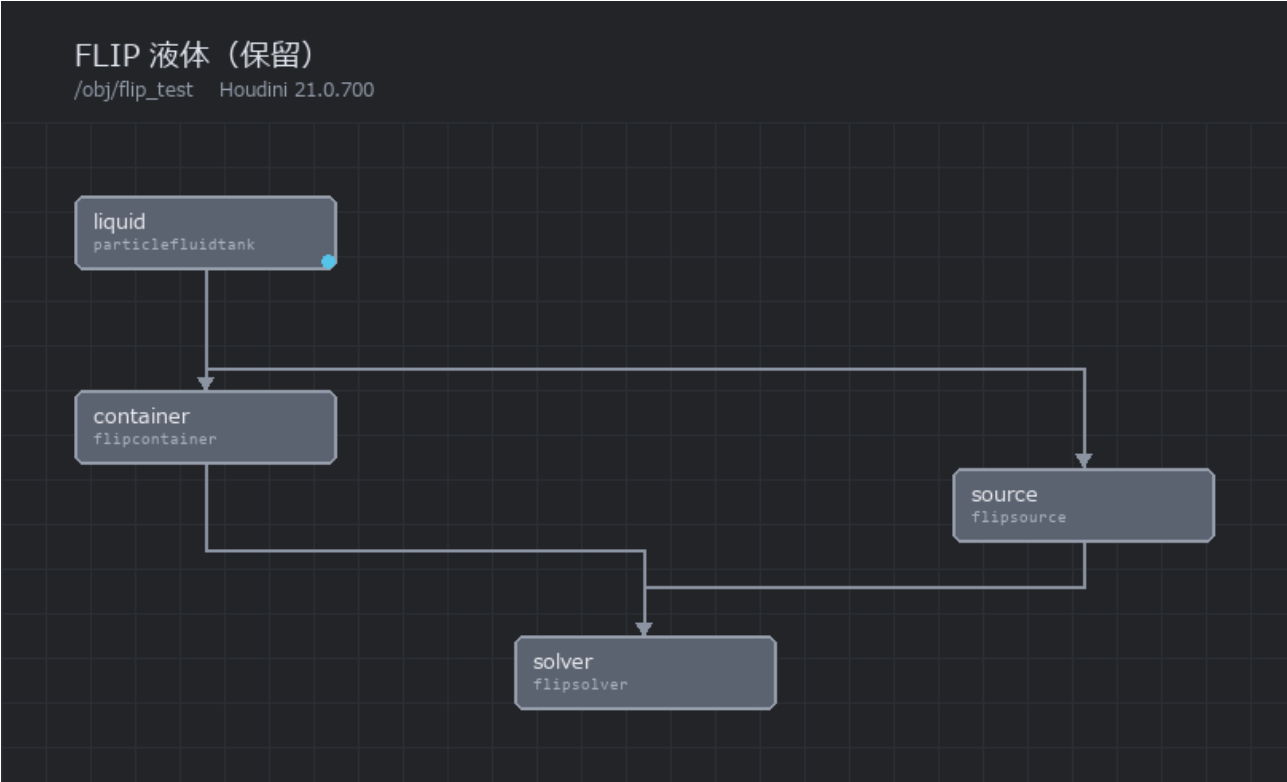
2026-09-13 | Houdini 21.0.700 | /obj/flip_test | ノード 4 / 接続 4

何を作ったか

液体は押し縮められないので体積が保たれるはず。FLIP は液体を粒で表す方式なので、粒1つが一定の体積を担うなら「粒の数」がそのまま体積にあたる。さらに FLIP には reseeding という粒を足したり間引いたりする仕組みがあり、これを切れば粒の数が完全に保たれるはずーという筋書きで実験を組んだ。

ところがソルバが動かなかった。4通りの配線を試したがすべて同じエラーで止まり、原因を特定できていない。ここまでの調査内容を残して保留とする。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
liquid	particlefluidtank	particlesep = 0.15 sizex = 2.0 sizey = 2.0 sizez = 2.0
container	flipcontainer	particlesep = 0.15
source	flipsource	particlesep = 0.15

ノード	タイプ	デフォルトから変更したパラメータ
solver	flipsolver	folder14 = 1 folder15 = 1 folder2 = 1 folder3 = 1 folder5 = 1 folder6 = 1 particlesep = 0.15 useground = 1 vis_ramp1cb = 1.0 vis_ramp2cb = 1.0 vis_ramp2cg = 0.5 vis_ramp2pos = 0.4993646740913391 …ほか 5 件

試した配線と結果

すべて flipsolver が「Not enough sources specified.」というエラーを返した。出力2には入力の粒がそのまま出てくるが、出力1（本来の結果）は空のまま。

配線	出力1	出力2	出力3	エラー
タンクの粒 → 入力1	なし	0点	0点	Not enough sources specified.
コンテナ → 入力1	なし	0点	0点	Not enough sources specified.
コンテナ → 入力1、粒 → 入力2	なし	1460点（素通し）	0点	Not enough sources specified.
コンテナ → 入力1、flipsource → 入力2	なし	1点（素通し）	0点	Not enough sources specified.

分かっていること

調べたこと	結果
particlefluidtank で粒は作れる	2×2×2、間隔0.15 で 1460粒
flipcontainer は単体で動く	出力1が7点、出力2が82点、出力3が2点。エラーなし
flipsource は単体で動く	1点1プリミティブ（ポリウム）。エラーなし
flipsolver の入力数	4つ（input1～input4）
flipsolver の出力数	3つ
source という名前のパラメータ	sourceband（既定 2.0）の1つだけ

わかったこと

- ・エラーメッセージは一貫して「Not enough sources specified.」。ソルバは入力のどこかに「液体の供給源」を期待しているが、粒を渡しても、コンテナを渡しても、flipsource を渡しても満たされなかった。
- ・他のソルバとの違いが大きい。rdbbulletsolver（実験014）も vellumsolver（実験015）も pyrosolver（実験016）も、入力1にジオメトリをつなぐだけで動いた。FLIP だけが追加の作法を要求している。

- ・パラメータ側にも手がかりが少ない。source を含む名前は sourceband ひとつしかなく、「どの入力供給源か」を指定する仕組みは見当たらない。つまり入力の順番や種類が決まっていて、そこに正しいものを渡す必要があるということ。

- ・途中で1つ罨を踏んだので記録しておく。particlefluidtank を上に移動させたら粒が0になった。このノードは「水位」を基準に中身を満たす仕組みで、タンクを水位より上に動かすと空になる。落下させたいときはタンクを動かすのではなく、地面のほうを下げる。

- ・この保留は失敗ではあるが、記録しておく価値がある。「他と同じやり方が通じないノードがある」と分かったこと自体が情報で、次に取り組むときに同じ4通りを試さずに済む。

- ・なお粒から見積もった体積は 4.92750 で、箱の体積 8.0 の 62% だった。粒は格子状に並ぶので隙間があり、粒の数 × 間隔の3乗では体積を正確には表せない。実際に検証に使うなら、この係数を先に求めておく必要がある。

次にやること

- ・Houdini 付属のヘルプ (nodes.zip) で flpsolver の入力の説明を読む。ネットではなく手元の一次情報から
- ・シェルフツールが作る FLIP のネットワークを GUI で作り、その構造を .hipnc から読み取る
- ・先に Karma でのレンダリングを片付ける。実験016でボリウムが描けないと分かっているため

Karma でのレンダリング – ノイズはサンプル数で本当に減るのか

2026-09-13 | Houdini 21.0.700 | /obj/pyro_test | ノード 6 / 接続 5

何を作ったか

実験016で煙を描いたとき、ハードウェアレンダラー（OpenGL ROP）では濃淡が分らないと書いた。そこで本式のレンダラーである Karma を使う。ただ「きれいになった」で終わらず、測れるものを測る。

Karma は1ピクセルの色を、乱数で飛ばした何本もの光線の平均で決める。平均のばらつきは本数の平方根に反比例するはずなので、サンプル数を4倍にすればノイズは半分になるという式が立つ。これを実測で確かめた。

結果、式が当たるのは狭い範囲だけだった。そして016で書いた「ハードウェアレンダラーはボリウムを描けない」も、言い過ぎだったと分かった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.5 rady = 0.5 radz = 0.5 ty = 0.6 type = 1
source	pyrosource	attribute1 = 0 attributes = 1 mode = 2
rasterize	volumerasterizeattributes	attributes = density folder1 = 1 folder3 = 1 voxelsize = 0.08

ノード	タイプ	デフォルトから変更したパラメータ
solve_d0_0	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 102 件
solve_d0_1	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 100 件
solve_d0_3	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.3 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 101 件

サンプル数を変えたときのノイズ

ノイズの量は、基準画との差ではなく、3枚を連立させて1枚ずつ求めた。基準画にもノイズが残っている
ので、基準画に頼ると結果がその出来に左右されるため。640×360、煙とみなした画素は 21,525 個。

サンプル数	ノイズ	前段との比	式の予測	ずれ
2	18.316	—	—	—
4	11.321	0.6181	0.7071	−12.6%
8	6.516	0.5756	0.7071	−18.6%
16	4.559	0.6997	0.7071	−1.0%
32	3.413	0.7486	0.7071	+5.9%
64	2.640	0.7734	0.7071	+9.4%
128	2.345	0.8883	0.7071	+25.6%

サンプル数	ノイズ	前段との比	式の予測	ずれ
256	2.088	0.8904	0.7071	+25.9%

サンプル数を4倍にしたとき（式なら 0.5000）

8→32 の区間だけが式どおり。その下では式より速く、その上では遅く減る。

区間	実測の比	式との差
2 → 8	0.3558	式より速く減る
4 → 16	0.4028	式より速く減る
8 → 32	0.5238	ほぼ式どおり
16 → 64	0.5789	やや遅い
32 → 128	0.6870	明らかに遅い
64 → 256	0.7909	ほとんど減らない

指定したサンプル数と、実際に使われたサンプル数

2～256 の範囲で時間を直線に当てはめると「固定費 2.26秒 + 1サンプルあたり 7.44ミリ秒」になる。この直線から逆算した実効サンプル数を右端に置いた。指定を64倍にしても、実際には6倍にもなっていない。

指定	実測の時間	直線の予測	実測/予測	実効サンプル
64	3.00 秒	2.74 秒	1.10	99
128	3.09 秒	3.21 秒	0.96	112
256	3.43 秒	4.16 秒	0.82	157
512	4.23 秒	6.07 秒	0.70	264
1024	5.52 秒	9.88 秒	0.56	439
2048	5.94 秒	17.50 秒	0.34	495
4096	6.48 秒	32.73 秒	0.20	567

頭打ちの犯人 — varianceaa_thresh

samplesperpixel を 4096 に固定したまま、打ち切りのしきい値だけを下げた。しきい値を50分の1にすると、実際の仕事量は21.9倍になった。頭打ちはハードウェアの限界ではなく、この設定によるもの。

varianceaa_thresh	時間	実効サンプル	既定との比
0.01（既定）	5.39 秒	420	1.00
0.005	6.29 秒	542	1.29
0.001	8.01 秒	773	1.84
0.0002	70.64 秒	9,191	21.87

同じ煙を2つのレンダラーで — 同じカメラ・同じ露出

「煙の画素」は真っ白でない画素の数。「最も暗い」は、その画素がどこまで濃く描かれたか。

レンダラー	時間	煙の画素	階調	平均の明るさ	ばらつき	最も暗い
OpenGL ROP	0.71 秒	29,242	235 段	101.0	86.16	15
Karma 256サンプル	3.64 秒	21,545	200 段	109.0	52.83	50

煙の中心を横切る線の明るさ

左から右へ等間隔に見たときの値。255 が背景。数字が小さいほど濃い。

レンダラー	縁の手前	縁	中へ	中心	中心	中へ	縁	縁の外
OpenGL ROP	254	224	121	58	42	50	112	219
Karma	255	255	184	109	104	104	189	255

わかったこと

- ・式が当たったのは 8~32 の区間だけだった。サンプル数を4倍にしたときの比は、8→32 で 0.5238（式は 0.5000）とよく一致する。ところが 64→256 では 0.7909 で、ほとんど減らない。

- ・減らなくなる理由を突き止めた。samplesperpixel

は「ここまで使ってよい」という上限であって、「必ずこれだけ使う」という指定ではない。指定を 64 から 4096 へ64倍にしても、時間から逆算した実効サンプル数は 99 から 567 へ5.7倍にしかならなかった。

- ・犯人は varianceaa_thresh（既定

0.01）だった。これは「このくらい収束したら、そのピクセルはもう打ち切る」というしきい値。4096 を指定したまましきい値を 0.0002 まで下げると、実効サンプル数は 420 から 9,191 へ21.9倍になり、時間も 5.39秒から70.64秒に伸びた。上限ではなく打ち切りの設定が効いていた。

- ・実用的な結論。この場面では 32 サンプルあたりで費用対効果が折り返す。それ以上は、数字を上げるだけでは絵は良くなる。本当に詰めたいときは varianceaa_thresh も一緒に下げる。ただし時間は一気に13倍になる。

- ・時間の内訳も分かった。640×360 で固定費が 2.26 秒、1サンプルあたり 7.44 ミリ秒。2サンプルでは時間のほぼ全部が固定費で、256サンプルでも半分強が固定費だった。軽い絵を何枚も撮るときは、サンプル数より枚数のほうが効く。

- ・実験016に書いた「ハードウェアレンダラーはボリュームを描けない」は言い過ぎだった。同じカメラ・同じ露出で撮り直すと、OpenGL ROP も煙の輪郭と、厚みによる濃淡（中心ほど濃い）を描けている。中心を横切る線を見ると、縁の 254 から中心の 42 までなだらかに変化している。

- ・2つのレンダラーの本当の違いは、光が中を通るかどうかが違った。OpenGL は濃さの分だけ暗くするだけで、どちらから光が当たっているかが絵に出ない。Karma は光源側が明るく、反対側が暗い。Karma のほうが最も暗い画素が 50 と薄いのも、光が内部を通過して抜けてくるため。

- ・速さの差は大きい。OpenGL が 0.71 秒に対し、Karma は 256サンプルで 3.64 秒。5倍。確認のための描画は今後も OpenGL でよく、Karma は見せる絵に使う。

- ・測り方で3回つまずいた。(1) 出力画像には透明度が付いていて、透明部分の色は黒だった。白い背景に見えるのは表示側が白で合成しているため。これに気づかず「白から離れた画素＝煙」というマスクを作り、画面の88%を拾って、ほとんど何も測っていなかった。

- ・ (2) 煙の芯が真っ白 (255) に張り付いていた。張り付いた画素は元の値が分からないので、ノイズを測っても意味がない。ライトを下げ、中間調の画素だけを対象にした。

- ・ (3) OpenGL と Karma でカメラの寄り方が違っていた (margin 1.12 対 0.5) 。煙の写る画素数が違って当たり前で、最初の比較は無効だった。実験003以降ずっと守ってきた「全変種を同じカメラで撮る」を、自分で破っていた。

- ・ Apprentice でも Karma

は問題なく動く。出力には右下にウォーターマークが入るので、測る範囲からは外している。

次にやること

- ・ 構造のある煙を作って撮り直す。いまの煙はほぼ一様な球で、レンダラーの差を見せる題材として弱い

- ・ 018で決めた Karma の設定 (denoiser off、varianceaa_thresh、露出) を hou_tools に関数としてまとめる

- ・ 019 として FLIP に再挑戦する。付属ヘルプで flipsolver の入力の作法を読む